VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



COMPUTER GRAPHICS

Assignment 1

Final Report

Instructor(s): Trần Thị Ngọc Trâm

Group: 5

Students: Lâm Mỹ Phương - 2352950
Lý Trần Tân - 2353077

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Introduction **2**

2 Overall Architecture and Shared Components **3**
2.1 Technology and repository layout 3
2.2 Setup, build and run 4
2.3 Shared components 5

3 Part 1: Drawing Basic Shapes **5**
3.1 Architecture and Components 6
3.2 Geometry: Vertices and Indices of Every Shape 7
3.3 Surface $z = f(x, y)$ and Imported Models 14
3.4 Rendering Modes and Shaders 16
3.5 Object Transformations 18
3.6 User Interface and User Instructions 19
3.7 Results 20
3.8 Limitations 22

4 Part 2: Visualization of Atoms and Molecules **22**
4.1 Architecture and Components 24
4.2 Mathematical Foundations 26
4.3 Visualizing Atoms: Bohr's Model 30
4.4 Visualizing Molecules: Ball-and-Stick Model 35
4.5 Animation (Optional) 39
4.6 Rendering and Lighting 41
4.7 User Interface and User Instructions 42
4.8 Testing and Verification 44
4.9 Results 44
4.10 Limitations 49
4.11 Limitations 49

5 Conclusion **50**
5.1 Part 1 50
5.2 Part 2 50

6 Source Code **51**

References **52**

1 Introduction

This report describes Assignment 1 of the Computer Graphics course (CO3059): *2D and 3D object rendering*. Two interactive graphical applications are implemented with modern, shader-based OpenGL:

- **Part 1 – Drawing Basic Shapes:** render a variety of 2D and 3D shapes (including a user-defined surface and imported models) with a GUI, camera control, geometric transformations and several rendering modes.
- **Part 2 – Visualization of Atoms and Molecules:** visualize atoms with Bohr's model and simple molecules with the ball-and-stick model.

Both applications are written in C++17 on top of OpenGL 3.3 (core profile), GLFW, GLEW, GLM and Dear ImGui [1, 4, 5, 6, 7], run on Linux, and share a small common code base.

Organisation of this report. Section 2 describes what is common to both parts (technology, repository, build instructions, shared classes, lighting model). Sections 3 and 4 describe Part 1 and Part 2. Section 5 concludes and gives the link to the source code.

Requirements and where they are met.

Requirement (assignment text)	Implementation
Part 1	
2D shapes: triangle, rectangle, pentagon, regular hexagon, circle, ellipse, trapezoid, star, arrow	Section 3.2: generators in `Geometry.cpp`
3D solids: cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism	Section 3.2
Surface $z = f(x, y)$ from a user-provided function	Section 3.3.1: expression parser + grid mesh
Imported model from `.obj` / `.ply`	Section 3.3.2: hand-written loaders
GUI with menus / toolbars to select the shape to add	Section 3.6: menu bar and inspector (Dear ImGui)
Mouse and/or keyboard zoom, pan, rotate	Section 2.3.2 (camera) and Section 3.6 (controls)
Flat color, Gouraud, Phong, texture, wireframe	Section 3.4: one shader program, five modes
Translation, rotation, scaling	Section 3.5
Clean, modular code	Section 3.1
Part 2	
GUI with menus / toolbars to toggle the features	Section 4.7: ImGui control panel with element / molecule selection
Visualize atoms using Bohr's model: electrons as small spheres orbiting the nucleus in distinct shells	Section 4.3: scene-graph-driven Bohr atom with tilted orbital shells
Visualize simple molecules (e.g. H_2O, CO_2): atoms as colored spheres, bonds as cylinders	Section 4.4: 8 predefined molecules, CPK colors, single / double / triple bonds
Optional: animate electron motion along orbitals; molecular vibration or rotation	Section 4.5: electron orbits, atom vibration, whole-molecule rotation
Camera control and screenshots of the molecule structures	Section 2.3.2 (shared orbit camera)

2 Overall Architecture and Shared Components

2.1 Technology and repository layout

- **Language / API:** C++17, OpenGL 3.3 core profile, GLSL 330, 4× MSAA.
- **Libraries:** GLFW (window, input), GLEW (extension loading), GLM (math), Dear ImGui (GUI). Part 1 additionally uses the single-header libraries `stb_image`

/ `stb_image_write` (image loading and screenshots) [8].

- **Shared code:** the `Shader` and `Camera` classes and the ImGui / GLM libraries live in `common/` and are compiled by both parts.

```
cg/
├── Makefile, README.md      build both parts; overview
├── common/                  shared by both parts
│   ├── src/                 Shader.h/.cpp, Camera.h/.cpp
│   └── vendor/              imgui, glm
├── part1_shapes/            Part 1
│   ├── src/                 application source
│   ├── shaders/             shapes.vert, shapes.frag
│   ├── assets/              sample textures (png) and models (obj, ply)
│   └── vendor/stb/          stb_image.h, stb_image_write.h
├── part2_molecules/         Part 2
│   ├── src/                 application source
│   └── shaders/             phong.vert, phong.frag
├── docs/                    assignment statement
└── report/                  this report (.tex, .pdf) and its figures
```

Each part has its own `Makefile` and builds into its own `build/` folder; the top-level `Makefile` calls both.

2.2 Setup, build and run

```
sudo apt install build-essential libglfw3-dev libglew-
dev    # Ubuntu / Debian

cd cg
make               # build both parts (or: make part1 / make part2)
make run1          # start Part 1 (make run2: Part 2)
make test          # Part 2 unit tests (no GPU needed)
make clean         # remove the build output of both parts
```

Dear ImGui, GLM and the stb headers are bundled in the repository, so no other dependency has to be installed. The programs must be started from their own folder (`part1_shapes` or `part2_molecules`) so that `shaders/` (and, for Part 1, `assets/`) are found; `make run1` / `make run2` do this. All data files Part 1 needs to run (sample textures and models) are delivered in `part1_shapes/assets/`.

2.3 Shared components

2.3.1 Shader class

`Shader` (`common/src/Shader.cpp`) loads a vertex and a fragment shader from text files, compiles each stage and links the program; compile or link errors are turned into an exception carrying the GLSL info log. `use()` activates the program and the typed setters `setMat4`, `setMat3`, `setVec3`, `setFloat`, `setInt`, `setBool` upload uniforms by name. Both parts keep their GLSL files in their own `shaders/` folder.

2.3.2 Camera

The `Camera` class (`common/src/Camera.cpp`) is an orbit camera defined by `yaw`, `pitch`, `distance` and a `target`:

$$\text{eye} = \text{target} + d\,\big(\cos\varphi\cos\psi,\ \sin\varphi,\ \cos\varphi\sin\psi\big),$$

with ψ=yaw and φ=pitch (clamped to $\pm 89°$). The view matrix is `lookAt(eye, target, up)` and the projection is a perspective matrix (default field of view 45°, near 0.1, far 100, adjustable). The class also implements the mouse interaction used by both applications: dragging the left button changes yaw / pitch (orbit), dragging the right button moves the target in the camera's right / up plane (pan, scaled by the distance), and the wheel changes the distance (zoom, clamped to $[1, 50]$). The applications ignore the mouse while it is over the ImGui panels.

2.3.3 Phong lighting model

Both applications light their objects with the Phong reflection model [2] and a single point light whose position and ambient / diffuse / specular colors are editable in the GUI. For a surface point P with unit normal N, light direction $L = \text{normalize}(\text{lightPos} - P)$, view direction $V = \text{normalize}(\text{viewPos} - P)$ and reflection $R = \text{reflect}(-L, N)$, the color is

$$C \;=\; L_a \odot m_a \;+\; L_d \odot m_d \,\max(N\cdot L,\,0) \;+\; L_s \odot m_s \,\max(R\cdot V,\,0)^{\,n},$$

where L_a, L_d, L_s are the light's ambient, diffuse and specular colors, m_a, m_d, m_s the material's, $\odot$ is the component-wise product and n is the shininess. The ambient term keeps the unlit side visible, the diffuse term gives the object its volume, and the specular term adds the bright highlight whose size is controlled by n. The way each part evaluates this formula (per vertex, per fragment) is described in its own section.

3 Part 1: Drawing Basic Shapes

Part 1 asks for an interactive application that renders basic 2D shapes (triangle, rectangle, pentagon, regular hexagon, circle, ellipse, trapezoid, star, arrow) and 3D shapes

(cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism, a surface $z = f(x, y)$ and an imported `.obj` / `.ply` model), with a GUI to choose the shape, mouse / keyboard zoom, pan and rotate, and five rendering modes (flat color, Gouraud, Phong, texture, wireframe).

3.1 Architecture and Components

3.1.1 Modules

The code is split so that everything that does not need OpenGL (geometry, parsing, file loading) is independent of the rendering code.

File(s)	Responsibility
`main.cpp`	Window, GLFW callbacks (mouse / keyboard), render loop, command-line options.
`App.h`	State shared by `main` and the GUI: `Scene`, `Camera`, light, camera presets.
`Gui.cpp`	Menu bar, object list, parameter / transform / appearance editors, import dialog.
`Scene.cpp`	Object list, selection, duplicate / delete, ground grid, axes, selection box, demo scene.
`SceneObject.cpp`	One shape: kind + parameters, transform, appearance; builds its mesh; issues the draw call.
`Geometry.cpp`	CPU mesh generators for all 2D / 3D shapes and mesh utilities (normals, colors, edge list). No OpenGL.
`ShapeMesh.cpp`	GPU buffers (VAO / VBO / two EBOs); `draw()` and `drawWire()`.
`Surface.cpp, ExprParser.cpp`	Mesh of $z = f(x, y)$ and the formula compiler.
`ModelLoader.cpp`	`.obj` and `.ply` importers.
`Texture.cpp`	Image loading (stb_image), mipmaps, per-file cache, built-in checkerboard.
`Screenshot.cpp, Paths.cpp`	PNG screenshot; locating shader / asset files.
`common/Shader.cpp`	Compiles and links GLSL, uniform setters.
`common/Camera.cpp`	Orbit camera: view / projection matrices, mouse rotate / pan / zoom.

3.1.2 Data flow

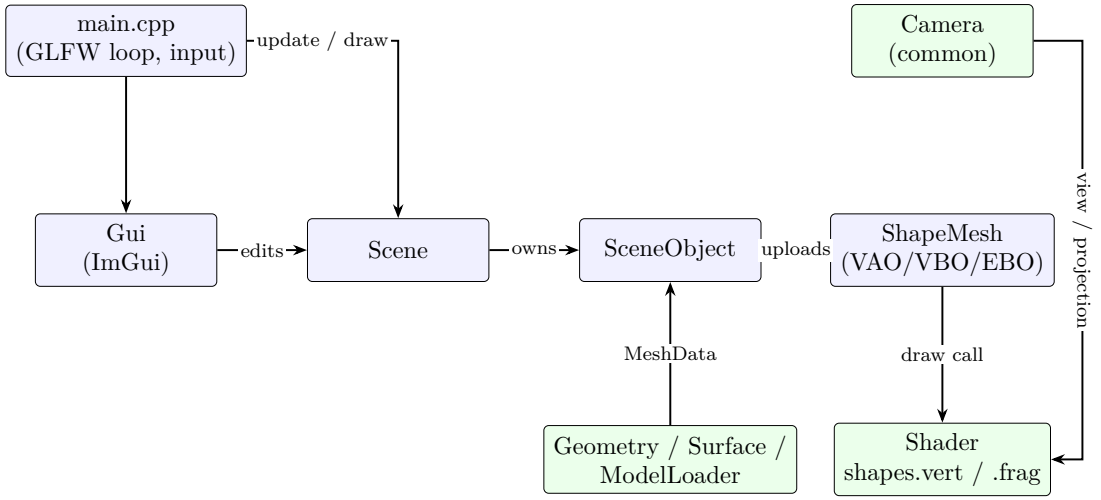


Figure 1: Main components (*blue*: application classes, *green*: data sources / GPU side). A `SceneObject` asks `Geometry` (or `Surface` / `ModelLoader`) for a `MeshData`, uploads it as a `ShapeMesh`, and every frame sets its uniforms and draws it with the shared shader.

Each frame, `main` (i) processes input, (ii) uploads the camera and light uniforms, (iii) lets the `Scene` draw the grid, every visible object and the selection box, (iv) draws the ImGui interface on top.

3.2 Geometry: Vertices and Indices of Every Shape

3.2.1 Conventions

Vertex format. Every mesh uses the same interleaved vertex (11 floats = 44 bytes):

Attribute	Type	Used for
`position` (location 0)	`vec3`	object-space position
`normal` (location 1)	`vec3`	unit normal, lighting
`color` (location 2)	`vec3`	per-vertex color (Gouraud mode, grid, axes)
`uv` (location 3)	`vec2`	texture coordinates

Index buffers. Each mesh owns two index lists (both `unsigned int`):

- `indices`: a *triangle list*, 3 indices per triangle, drawn with `GL_TRIANGLES`. Triangles are wound counter-clockwise when seen from outside (the direction of the vertex normal).

- `wireIndices`: a *line list*, 2 indices per edge, drawn with `GL_LINES` in wireframe mode (Section 3.2.10).

Notation. V = number of vertices, I = number of triangle indices, $T = I/3$ = number of triangles, E = number of wireframe edges. All shapes are centred at the origin and are about one unit large; 2D shapes lie in the XY plane with normal $+Z$, 3D solids use Y as the up axis.

Why vertices are duplicated. A vertex is the unit stored in the vertex buffer: one position *together with* one normal, one color and one UV. Whenever two faces meet along a sharp edge (cube, tetrahedron, prism sides, cylinder caps) each face needs its own normal, so the corner is stored once per face — e.g. a cube has $6 \times 4 = 24$ vertices, not 8. Likewise the first and last column of a sphere or torus have the same position but different u texture coordinates (0 and 1), so the seam is stored twice. Using indices lets the triangles of a face share the vertices of that face.

Parameters. The "default" columns below use the defaults of the GUI: 48 segments for round shapes, 24 stacks (sphere) / tube segments (torus), 6 prism sides, 5 star points, surface resolution 64.

3.2.2 Summary

Shape	**Parameters**	V	I	T	E	**Defaults** $(V/I/E)$
Triangle	$n = 3$	$n+1$	$3n$	n	$2n$	4 / 9 / 6
Rectangle	—	5	12	4	8	5 / 12 / 8
Pentagon	$n = 5$	$n+1$	$3n$	n	$2n$	6 / 15 / 10
Hexagon	$n = 6$	$n+1$	$3n$	n	$2n$	7 / 18 / 12
Circle	N segments	$N+1$	$3N$	N	$2N$	49 / 144 / 96
Ellipse	N segments	$N+1$	$3N$	N	$2N$	49 / 144 / 96
Trapezoid	—	5	12	4	8	5 / 12 / 8
Star	p points	$2p+1$	$6p$	$2p$	$4p$	11 / 30 / 20
Arrow	—	7	15	5	11	7 / 15 / 11
Cube	—	24	36	12	18	24 / 36 / 18
Sphere	S sectors, K stacks	$(S+1)(K+1)$	$6S(K-1)$	$2S(K-1)$	$3S(K-1)$	1225 / 6624 / 3312
Cylinder	N segments	$4N+6$	$12N$	$4N$	$6N$	198 / 576 / 288
Cone	N segments	$3N+4$	$9N$	$3N$	$3N$	148 / 432 / 144
Truncated cone	N segments	$4N+6$	$12N$	$4N$	$6N$	198 / 576 / 288
Tetrahedron	—	12	12	4	6	12 / 12 / 6
Torus	M ring, m tube	$(M+1)(m+1)$	$6Mm$	$2Mm$	$3Mm$	1225 / 6912 / 3456
Prism	n sides	$6n+4$	$12n$	$4n$	$6n$	40 / 72 / 36
Surface	$N \times N$ cells	$(N+1)^2$	$\leq 6N^2$	$\leq 2N^2$	$\leq 3N^2+2N$	4225 / 24576 / 12416†
Model	from file	file	$3\sum(k_f-2)$	—	from file	see Section 3.3.2

Table 1: Vertex, index, triangle and edge counts. These closed forms are derived in the following subsections and were checked against the real generators with a program that builds each shape for many parameter values. † for $f = \sin x \cos y$ on $[-3, 3]^2$, where every cell is finite.

3.2.3 2D shapes: triangle fans

All 2D shapes except the arrow are *convex or star-shaped around the origin*, so they are triangulated as a **triangle fan** around a centre vertex $c = (0, 0, 0)$: given a counter-clockwise ring of n boundary points $p_0, \ldots, p_{n-1}$,

$$\begin{aligned} \text{vertices:} \quad & v_0 = c, \quad v_{1+i} = p_i \quad (i = 0, \ldots, n-1) \qquad \Rightarrow \qquad V = n+1, \\ \text{triangles:} \quad & \big(0,\ 1+i,\ 1+((i+1) \bmod n)\big), \quad i = 0, \ldots, n-1 \qquad \Rightarrow \qquad I = 3n. \end{aligned}$$

Every vertex has normal $(0, 0, 1)$. Texture coordinates use planar mapping of the bounding box: $u = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$, $v = \frac{y - y_{\min}}{y_{\max} - y_{\min}}$. Only the ring changes between shapes:

Shape	**Ring points** p_i	n
Triangle, pentagon, hexagon, circle	$R\,(\cos\alpha_i,\ \sin\alpha_i)$ with $\alpha_i = \frac{\pi}{2} + \frac{2\pi i}{n}$, $R = 0.5$ (first vertex points up)	$3, 5, 6, N$
Ellipse	$(r_x \cos\alpha_i,\ r_y \sin\alpha_i)$, $\alpha_i = \frac{2\pi i}{N}$ (default $r_x = 0.5$, $r_y = 0.3$)	N
Rectangle $w \times h$	$(-\frac{w}{2}, -\frac{h}{2})$, $(\frac{w}{2}, -\frac{h}{2})$, $(\frac{w}{2}, \frac{h}{2})$, $(-\frac{w}{2}, \frac{h}{2})$	4
Trapezoid (b, t, h)	$(-\frac{b}{2}, -\frac{h}{2})$, $(\frac{b}{2}, -\frac{h}{2})$, $(\frac{t}{2}, \frac{h}{2})$, $(-\frac{t}{2}, \frac{h}{2})$	4
Star with p points	$r_i(\cos\alpha_i, \sin\alpha_i)$, $\alpha_i = \frac{\pi}{2} + \frac{\pi i}{p}$, $r_i = R$ for even i and ρR for odd i ($R = 0.5$, inner ratio $\rho = 0.4$)	$2p$

Example: triangle ($n = 3$, $V = 4$, $I = 9$).

index	x	y	role
0	0	0	centre
1	0	0.5	$\alpha = 90°$
2	−0.433	−0.25	$\alpha = 210°$
3	0.433	−0.25	$\alpha = 330°$

indices: $(0, 1, 2)$, $(0, 2, 3)$, $(0, 3, 1)$

Example: rectangle 1×0.7 **($V = 5$, $I = 12$).** Vertices $0 = (0, 0)$, $1 = (-0.5, -0.35)$, $2 = (0.5, -0.35)$, $3 = (0.5, 0.35)$, $4 = (-0.5, 0.35)$; indices $(0, 1, 2)$, $(0, 2, 3)$, $(0, 3, 4)$, $(0, 4, 1)$.

Arrow (concave, $V = 7$, $I = 15$). A fan from the centre would cover points outside the arrow, so the arrow is triangulated by hand with seven shared vertices (the head reuses the two right-hand vertices of the shaft):

index	0	1	2	3	4	5	6
(x, y)	$(-.5, -.15)$	$(.1, -.15)$	$(.1, .15)$	$(-.5, .15)$	$(.1, -.4)$	$(.5, 0)$	$(.1, .4)$
part		shaft				head	

Indices: shaft $(0, 1, 2), (0, 2, 3)$; head $(4, 5, 1), (1, 5, 2), (2, 5, 6)$ — five triangles, 15 indices.

3.2.4 Cube

The cube of side $s = 1$ ($h = s/2$) is built face by face. Each face has an outward normal n and two tangent axes u, v with $u \times v = n$ (so the winding is counter-clockwise from outside). Its four vertices are

$h\,(n{-}u{-}v)$, $h\,(n{+}u{-}v)$, $h\,(n{+}u{+}v)$, $h\,(n{-}u{+}v)$, UV $(0, 0), (1, 0), (1, 1), (0, 1)$,

all with normal n, and the two triangles are $(b{+}0, b{+}1, b{+}2)$ and $(b{+}0, b{+}2, b{+}3)$ where $b = 4f$ is the index of the face's first vertex. The six faces:

face	n	u	v
$+X$	$(1,0,0)$	$(0,0,-1)$	$(0,1,0)$
$-X$	$(-1,0,0)$	$(0,0,1)$	$(0,1,0)$
$+Y$	$(0,1,0)$	$(1,0,0)$	$(0,0,-1)$
$-Y$	$(0,-1,0)$	$(1,0,0)$	$(0,0,1)$
$+Z$	$(0,0,1)$	$(1,0,0)$	$(0,1,0)$
$-Z$	$(0,0,-1)$	$(-1,0,0)$	$(0,1,0)$

$V = 6 \cdot 4 = 24$, $I = 6 \cdot 6 = 36$. For example the $+Z$ face (indices 16–19) has positions $(-.5,-.5,.5)$, $(.5,-.5,.5)$, $(.5,.5,.5)$, $(-.5,.5,.5)$ and triangles $(16,17,18)$, $(16,18,19)$.

3.2.5 Sphere

A UV sphere of radius $r = 0.5$ with S sectors (longitude) and K stacks (latitude). Vertex (i, j), $i = 0..K$, $j = 0..S$, is stored at index $i(S+1)+j$:

$$\begin{aligned} \varphi_i &= \tfrac{\pi}{2} - \tfrac{\pi i}{K}, \qquad \theta_j = \tfrac{2\pi j}{S}, \\ n_{ij} &= (\cos\varphi_i \cos\theta_j,\ \sin\varphi_i,\ \cos\varphi_i \sin\theta_j), \qquad p_{ij} = r\, n_{ij}, \\ (u,v)_{ij} &= \left(\tfrac{j}{S},\ 1 - \tfrac{i}{K}\right). \end{aligned}$$

For the normals of a sphere centred at the origin the normal equals the normalised position, so no extra computation is needed. There are $K+1$ rows and $S+1$ columns ($j = S$ duplicates $j = 0$ for the texture seam), hence $V = (S+1)(K+1)$.

Each quad between rows $i, i{+}1$ and columns $j, j{+}1$ is split in two triangles. With $k_1 = i(S+1)+j$ and $k_2 = k_1 + S + 1$:

```
1 if (i != 0)        { push(k1,   k1 + 1, k2);   }  // skipped at the north pole
    (degenerate)
2 if (i != stacks - 1) { push(k1 + 1, k2 + 1, k2); } // skipped at the south pole
```

Counting: K rows of S quads give $2SK$ triangles, minus the S degenerate triangles at each pole: $T = 2SK - 2S$, so $I = 3T = 6S(K-1)$. (Default $S = 48, K = 24$: $V = 1225$, $I = 6624$.)

3.2.6 Cylinder, cone and truncated cone

All three are the same generator `frustum(`r_b`,` r_t`,` h`,` N`)` with bottom radius r_b, top radius r_t, height h $(= 1)$ and N segments (cylinder $r_t = r_b = 0.5$, cone $r_t = 0$, truncated cone $r_t = 0.25$ by default).

Side. For $i = 0..N$ with $a_i = 2\pi i/N$ two vertices are created (bottom at index $2i$, top at $2i{+}1$):

$$\text{bottom} = (r_b \cos a_i,\ -\tfrac{h}{2},\ r_b \sin a_i), \quad \text{top} = (r_t \cos a_i,\ \tfrac{h}{2},\ r_t \sin a_i),$$
$$n_i = \text{normalize}\big(h \cos a_i,\ r_b - r_t,\ h \sin a_i\big), \qquad (u, v) = (i/N,\ 0) \text{ and } (i/N,\ 1).$$

The slanted normal (the y-component $r_b - r_t$) is what makes the cone and truncated cone shade correctly; for the cylinder it reduces to $(\cos a_i, 0, \sin a_i)$. For segment $i < N$ with $b_0 = 2i$, $t_0 = 2i + 1$, $b_1 = 2i + 2$, $t_1 = 2i + 3$: triangles (b_0, t_0, b_1) and (b_1, t_0, t_1). So the side has $2(N + 1)$ vertices and $2N$ triangles (for the cone, N of them are degenerate because $t_0 = t_1$ is the apex; they are harmless and are ignored in wireframe).

Caps. A cap is added only if its radius is positive. A cap has its own vertices (its normal is $\pm Y$, not the side normal): one centre plus $N + 1$ ring vertices $(r \cos a_i, \pm\frac{h}{2}, r \sin a_i)$ with $uv = (\frac{1}{2} + \frac{1}{2}\cos a_i,\ \frac{1}{2} + \frac{1}{2}\sin a_i)$, and N triangles $(c,\ p_i,\ p_{i+1})$ for the bottom cap and $(c,\ p_{i+1},\ p_i)$ for the top cap.

Totals.

	V	T	I
cylinder / truncated cone (2 caps)	$2(N{+}1) + 2(N{+}2) = 4N + 6$	$2N + 2N = 4N$	$12N$
cone (bottom cap only)	$2(N{+}1) + (N{+}2) = 3N + 4$	$2N + N = 3N$	$9N$

3.2.7 Tetrahedron

A regular tetrahedron of edge $e = 1.2$ is inscribed in a cube: with $k = e/(2\sqrt{2}) \approx 0.4243$ its corners are

$$P_0 = k(1, 1, 1),\ P_1 = k(1, -1, -1),\ P_2 = k(-1, 1, -1),\ P_3 = k(-1, -1, 1),$$

(every pair is $2\sqrt{2}\,k = e$ apart). Each of the 4 faces $(P_0P_1P_2)$, $(P_0P_3P_1)$, $(P_0P_2P_3)$, $(P_1P_3P_2)$ gets its own three vertices carrying the face normal $n = \text{normalize}\big((b - a) \times (c - a)\big)$; if $n \cdot (a + b + c) < 0$ the normal points inward, so b and c are swapped and n is negated. UV are $(0, 0), (1, 0), (\frac{1}{2}, 1)$. Indices: face f uses $(3f, 3f{+}1, 3f{+}2)$. $V = 4 \cdot 3 = 12$, $I = 12$.

3.2.8 Torus

A torus with ring radius $R = 0.5$ and tube radius r (default 0.18), M segments around the ring and m around the tube. Vertex (i, j), $i = 0..M$, $j = 0..m$ at index $i(m + 1) + j$, with $\theta_i = 2\pi i/M$, $\phi_j = 2\pi j/m$:

$$p_{ij} = \big((R + r\cos\phi_j)\cos\theta_i,\quad r\sin\phi_j,\quad (R + r\cos\phi_j)\sin\theta_i\big),$$
$$n_{ij} = (\cos\phi_j \cos\theta_i,\ \sin\phi_j,\ \cos\phi_j \sin\theta_i), \qquad (u, v) = (i/M,\ j/m).$$

The normal is the direction from the tube's centre circle to the surface point. With $a = i(m+1)+j$ and $b = a+m+1$ the quad is split into (a, a+1, b) and (b, a+1, b+1). $V = (M+1)(m+1)$, $T = 2Mm$, $I = 6Mm$.

3.2.9 Prism

A regular n-gon prism (radius 0.5, height 1, axis Y). Each side face i lies between angles $a_0 = 2\pi i/n$ and $a_1 = 2\pi(i+1)/n$ and has its own four vertices (flat normal $= (\cos\frac{a_0+a_1}{2}, 0, \sin\frac{a_0+a_1}{2})$), in the order bottom-$a_0$, top-$a_0$, bottom-$a_1$, top-$a_1$ with triangles $(0,1,2)$ and $(2,1,3)$ (offset by $4i$). The two caps are the same disc caps as for the cylinder, built with $N = n$. Therefore $V = 4n + 2(n+2) = 6n+4$ and $T = 2n + n + n = 4n$, $I = 12n$.

3.2.10 Wireframe edges

In wireframe mode the object is shown as its *triangle mesh*: every edge of every triangle is drawn as a line, so the triangulation built in the previous subsections becomes visible (the diagonal that splits each face of the cube, the radial lines of a fan or a cap, the quad diagonals of a sphere or torus). Each mesh keeps a separate line list `wireIndices`, built once by `buildWireIndices()`:

1. Vertices with the same position (quantised to 10^{-4}) get the same *canonical id*, so the duplicated vertices of a cube corner or of a texture seam are recognised as one point and their shared edge is drawn once, not twice.
2. Every non-degenerate triangle contributes its 3 edges; an edge is stored (as a pair of vertex indices for `GL_LINES`) the first time its pair of canonical ids is met. Degenerate triangles (sphere poles, cone apex) are skipped.

Since a triangle mesh is drawn, E follows from the triangulation; the counts are the E column of Table 1:

- **2D fans:** n boundary sides plus n radial lines to the centre, $E = 2n$ (star 4p, rectangle and trapezoid 8); the arrow has its 7 outline edges plus 4 interior diagonals, $E = 11$.
- **Cube:** 12 edges plus one diagonal per face, $E = 18$; **tetrahedron:** $E = 6$.
- **Cylinder / truncated cone:** N vertical lines, N side diagonals, 2 rims of N segments and $2N$ cap radials, $E = 6N$; **cone:** N slant lines, N rim segments and N base radials, $E = 3N$; **prism:** likewise $E = 6n$.
- **Sphere:** SK meridian segments, $S(K-1)$ latitude segments and $S(K-2)$ quad diagonals, $E = 3S(K-1)$; **torus:** $3Mm$ (two grid directions and one diagonal per quad).

- **Surface:** $2N(N+1)$ grid lines plus N^2 diagonals, $E = 3N^2 + 2N$ when every cell is finite.

3.2.11 Normals, colors and texture coordinates

- **Normals:** analytic for sphere, torus, cylinder / cone, flat per face for cube / tetrahedron / prism, analytic gradient for surfaces, area-weighted vertex averages for imported models that have no normals.
- **Vertex colors** (used by Gouraud mode): by default a rainbow computed from the normalised position inside the bounding box, $t = \text{mean}\left(\frac{x-x_{\min}}{\Delta x}, \frac{y-y_{\min}}{\Delta y}, \frac{z-z_{\min}}{\Delta z}\right)$ over the non-flat axes and hue $= 0.83\,t$; optionally a two-color gradient $\text{mix}(A, B, t)$ chosen in the GUI. For surfaces t is the height.
- **UV:** sphere and torus by their parameters (j/S etc.), cylinder / prism by angle and height, caps by a circle mapped to $[0,1]^2$, cube and tetrahedron per face, 2D shapes by their bounding box.

3.3 Surface $z = f(x, y)$ and Imported Models

3.3.1 Mathematical surface

The user types a formula such as `sin(x)*cos(y)` and a range $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ with a grid of $N \times N$ cells (default $N = 64$, up to 400).

Formula parser. `ExprParser.cpp` is a recursive-descent parser that compiles the text into a small expression tree evaluated for every grid point. Grammar (highest precedence last):

$$\begin{aligned} \textit{expr} &::= \textit{term}\ ((\texttt{+}|\texttt{-})\ \textit{term})^* & \textit{unary} &::= (\texttt{-}|\texttt{+})\ \textit{unary} \mid \textit{power} \\ \textit{term} &::= \textit{unary}\ ((\texttt{*}|\texttt{/})\ \textit{unary})^* & \textit{power} &::= \textit{primary}\ (\texttt{\^{}}\ \textit{unary})? \\ \textit{primary} &::= \text{number} \mid x \mid y \mid \pi \mid e \mid \textit{name}\,(\textit{expr}[,\textit{expr}]) \mid (\textit{expr}) \end{aligned}$$

Power is right-associative and binds tighter than unary minus ($-x^2 = -(x^2)$, $2^{3^2} = 512$). Implicit multiplication (`2x`, `3sin(x)`) and a leading `z=` are accepted. Functions: `sin cos tan asin acos atan sinh cosh tanh sqrt abs exp log ln floor ceil` and the two-argument `pow atan2 min max`. Errors (unknown name, missing parenthesis, wrong argument count) are reported in the GUI together with the position and the previous mesh is kept.

Vertices. For $i, j = 0..N$: $x_i = x_{\min} + i\,\Delta x$, $y_j = y_{\min} + j\,\Delta y$, $z_{ij} = f(x_i, y_j)$. The surface is centred in x, y and uniformly scaled by $s = 1/\max(\Delta X, \Delta Y)$ so that its footprint is about one unit wide (it can be scaled like any other object). The mathematical z-axis

becomes world "up" $(+Y)$ and the mathematical y becomes $-Z$ so that the coordinate system stays right-handed:

$$p_{ij} = \big(s\,(x_i - c_x),\ s\,z_{ij},\ -s\,(y_j - c_y)\big), \qquad n_{ij} = \text{normalize}(-f_x,\ 1,\ f_y),$$

where f_x, f_y are central differences of f ($h = 10^{-4}$ of the range), and $(u, v) = (i/N,\ j/N)$. $V = (N+1)^2$, stored at index $a = j(N+1) + i$.

Indices. For every cell with corners $a = (i, j)$, $b = (i{+}1, j)$, $c = (i, j{+}1)$, $d = (i{+}1, j{+}1)$ the triangles are (a, b, c) and (b, d, c) (counter-clockwise seen from $+Y$). A cell is skipped when any corner has f non-finite or $|f| > 10^3$ (e.g. $\log x$ for $x \le 0$, $1/x$ at 0), so $I = 6N^2$ for a fully finite function and less otherwise. If no cell is finite the surface is rejected with an error message.

3.3.2 Importing `.obj` and `.ply` models

Both loaders are written by hand (no external model library) and fill the same `MeshData`.

OBJ. Lines `v`, `vt`, `vn` fill three arrays; each `f` corner `v/vt/vn` (any of `v`, `v/vt`, `v//vn`, `v/vt/vn`; negative indices allowed) is mapped to a GPU vertex. A vertex is the *unique triple* (position, uv, normal) — a lookup table reuses an existing vertex when the same triple appears again — so a vertex shared by many faces is stored once, but a vertex with two different normals or UVs is split. A face with k_f corners becomes a triangle fan (f_0, f_i, f_{i+1}), $i = 1..k_f - 2$, thus $I = 3\sum_f (k_f - 2)$. (Fan triangulation is exact for the convex faces found in typical models.)

PLY. The header is parsed generically (`ascii`, `binary_little_endian`, `binary_big_endian`; all scalar types; list properties). For the `vertex` element the properties `x y z`, `nx ny nz`, `s t`/`u v`, `red green blue` are used, and for `face` the list `vertex_indices` is fan-triangulated as above. Unknown elements and properties are skipped.

Post-processing. The model is re-centred on its bounding-box centre and scaled so that its largest extent is 1; normals are generated (area-weighted, merged by position) if the file has none; spherical UV are generated if it has none; vertex colors are kept when present (PLY) and rainbow otherwise; finally the wire edge list is built. Samples (`assets/models/`): `torusknot.obj` (3417 vertices, 6400 triangles, with normals and UV), `octahedron.obj` (6 vertices, 8 triangles, positions only), `icosphere_colored.ply` (ascii, 42 vertices, 80 triangles, colors) and `cube_binary.ply` (binary, 8 vertices, quads $\to$ 12 triangles).

3.4 Rendering Modes and Shaders

All objects are drawn by *one* shader program (`shapes.vert`, `shapes.frag`). The integer uniform `mode` chosen per object selects the mode:

`mode`	Render mode	Color computation
0	Flat color	`objectColor`, no lighting (a single uniform color).
1	Gouraud	Lighting evaluated per *vertex* in the vertex shader from the vertex color; the resulting color is interpolated across the triangle.
2	Phong	Normal interpolated; lighting evaluated per *fragment* with `objectColor`.
3	Texture	Same lighting as Phong, but the albedo is `texture(tex, uv*uvScale)`.
4	Wireframe	`objectColor`, drawn with `GL_LINES` from `wireIndices`.
5	(internal)	Unlit vertex color: ground grid and axes.

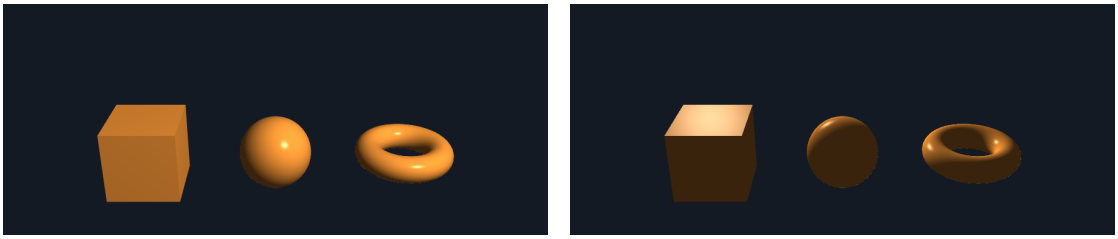
Lighting model. Modes 1–3 use the Phong reflection model of Section 2.3.3 with one point light. In Part 1 the material is reduced to three numbers per object: the *albedo* c (the object color, the vertex color or the texel), which serves as both m_a and m_d, the *Specular* slider (m_s is a gray value) and the *Shininess* (n). The lighting is *two-sided*: if $N \cdot V < 0$ the normal is flipped, so the back of a 2D shape and meshes with inconsistent winding are lit correctly (face culling is disabled).

Listing 1: Core of the lighting function (both shaders).

```
vec3 phong(vec3 N, vec3 P, vec3 albedo) {
    vec3 V = normalize(viewPos - P);
    if (dot(N, V) < 0.0) N = -N;                // two-sided lighting
    vec3 L = normalize(light.position - P);
    vec3 ambient = light.ambient * albedo;
    float diff   = max(dot(N, L), 0.0);
    vec3 diffuse = light.diffuse * diff * albedo;
    vec3 R       = reflect(-L, N);
    float spec   = diff > 0.0 ? pow(max(dot(V, R), 0.0), shininess) : 0.0;
    return ambient + diffuse + light.specular * spec * specularStrength;
}
```

Gouraud vs. Phong. In Gouraud mode [3] `phong()` is called in the *vertex* shader with the vertex color as albedo and the result is passed as an interpolated `out` variable; the fragment shader only outputs it. In Phong mode the vertex shader passes the world position and normal, and the fragment shader calls `phong()` for every pixel. The difference is visible in the highlight on low-resolution meshes: Gouraud can miss or smear a highlight that falls inside a triangle.

Highlights on flat faces. A curved surface always has some point whose normal reflects the light straight to the eye, so a sphere or torus shows a bright spot. A flat face has a single normal, hence a single reflection direction: the highlight is either one broad bright area (if the mirror point of the light falls on the face) or absent (if it falls outside the face, as with the default light on the cube in Figure 2a). With the light placed so that the mirror point lies on the top face (Figure 2b) the cube shows its highlight. This is the expected behaviour of the Phong model and not a rendering error; lowering the shininess widens the highlight.



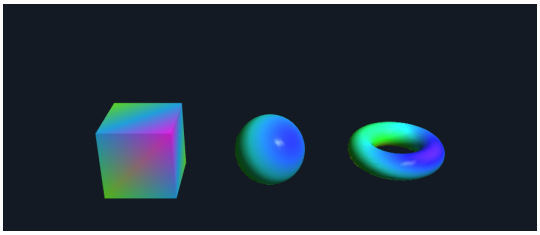
(a) Default light $(5, 8, 6)$ (b) Light $(-3.34, 3.89, -6.19)$

Figure 2: Phong highlight on a flat face depends on the light position (same objects, same camera).

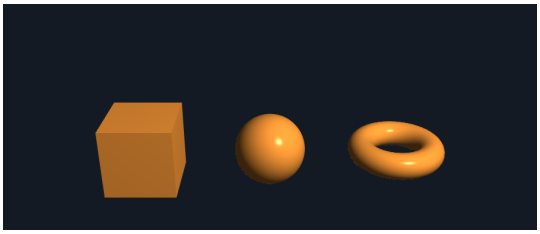
Texture. Images are loaded with `stb_image` (rows flipped so that $v = 0$ is the bottom), uploaded as RGBA8 with generated mipmaps, `GL_REPEAT` wrapping and trilinear filtering. A *Tiling* slider multiplies the UV. If no image is chosen, a built-in checkerboard is used.



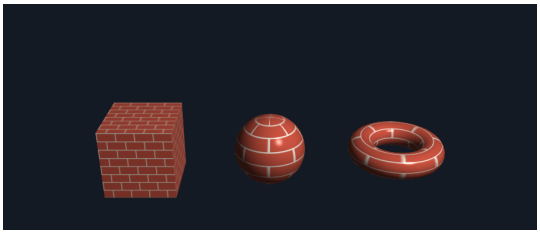
(a) Flat color



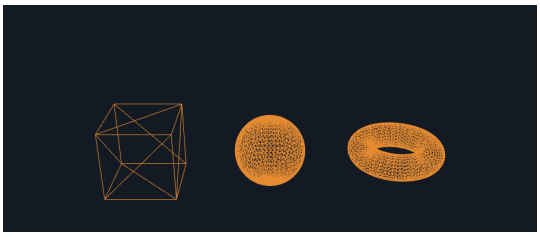
(b) Gouraud (rainbow vertex colors)



(c) Phong



(d) Texture (bricks.png)



(e) Wireframe

Figure 3: The five render modes on a cube, a sphere and a torus.

3.5 Object Transformations

Each object has a translation t, Euler rotation angles $(\theta_x, \theta_y, \theta_z)$ in degrees and a (non-uniform) scale s. The model matrix is

$$M = T(t)\; R_z(\theta_z)\, R_y(\theta_y)\, R_x(\theta_x)\; S(s),$$

i.e. a vertex is first scaled, then rotated about X, Y, Z and finally translated. M is uploaded as the `model` uniform, and normals are transformed with the normal matrix $\left(M^{-1}\right)^T$ (upper 3×3), which stays correct under non-uniform scaling. The inspector exposes these as sliders and shows the current 4×4 matrix. *Ctrl + left drag* in the viewport rotates the selected object, and *Reset transform* returns it to the centre.

3.6 User Interface and User Instructions

3.6.1 Using the application

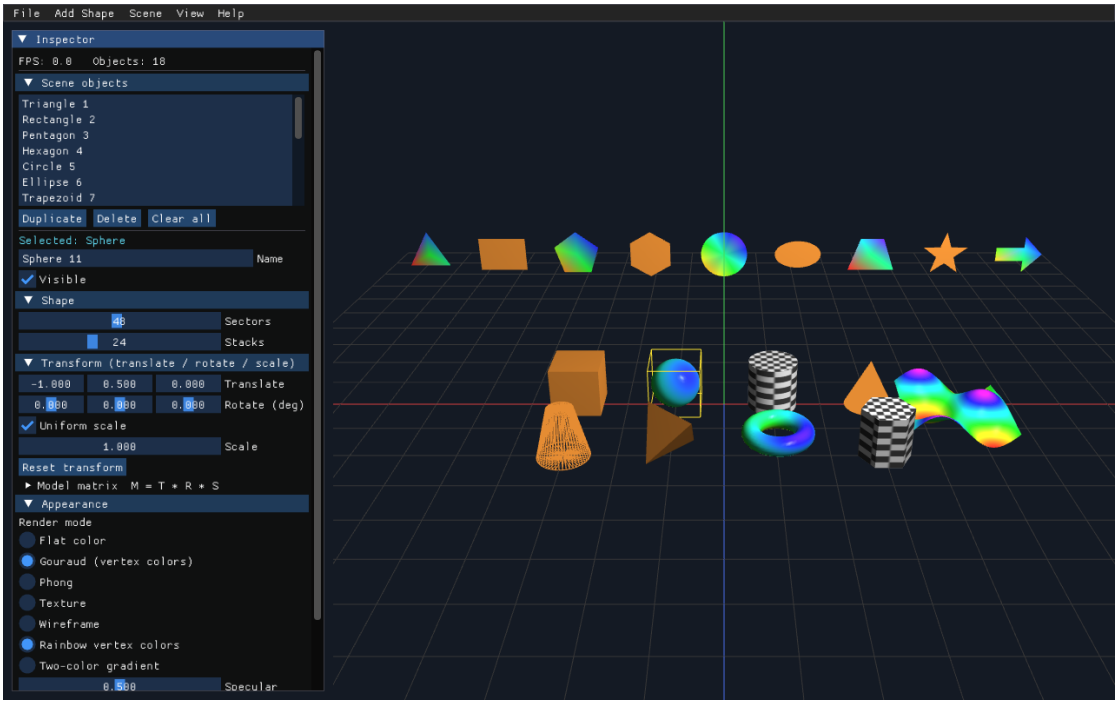


Figure 4: The interface: menu bar (top), inspector (left) and the 3D viewport with the demo scene. The yellow box marks the selected object.

- **Add Shape** menu: choose a 2D shape, a 3D shape, *Surface* $z = f(x, y)$ or *Import model…*. The new shape appears at the centre and becomes selected.
- **Inspector** → *Scene objects*: select, duplicate, delete or clear objects. *Shape*: parameters of the selected shape (segments, star points, inner ratio, top radius, tube radius, prism sides, rectangle size, …). For a surface, the formula, presets, ranges and resolution.
- *Transform*: translate / rotate / scale; *Appearance*: render mode, color, specular and shininess, vertex-color scheme (Gouraud), texture image and tiling.
- **Scene** menu: delete / duplicate, clear, load a demo scene with all shapes, set all objects to one mode. **View** menu: reset camera, front (2D) view, top view, grid, axes. **File**: import a model, save a screenshot.
- **Lighting** (light position and colors) and **Camera / View** (field of view, distance, background) at the bottom of the inspector.

Input	Action
Left mouse drag	orbit the camera
Right mouse drag	pan
Mouse wheel	zoom
Ctrl + left drag	rotate the selected object
Arrow keys / Shift + arrows	orbit / pan
`+ -` or PageUp / PageDown	zoom
`R`	reset camera
`W`	toggle wireframe (selected object, or all if none is selected)
`Del`	delete the selected object
`F12`	save a screenshot (`screenshot_<time>.png`)
`H` / `Esc`	help window / quit

Command line (for scripted screenshots). All figures in this report were produced with options such as

```
./build/shapes --add star --add torus --mode phong --screenshot out.png
```

Available options: `--demo`, `--add <shape>`, `--mode flat|gouraud|phong|texture|wire`, `--surface "<formula>"`, `--model <file>`, `--texture <png>`, `--view front|top`, `--distance d`, `--select i`, `--no-grid`, `--with-gui`.

3.7 Results

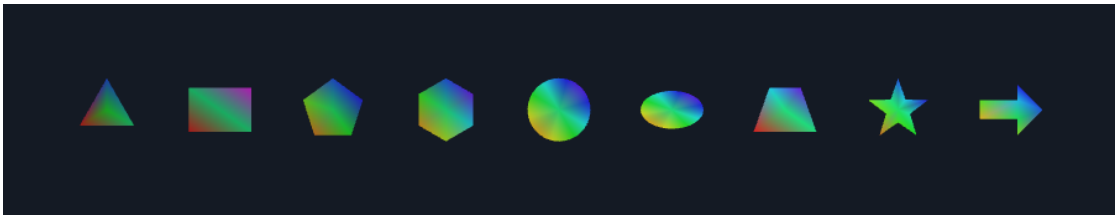


Figure 5: 2D shapes (front view, Gouraud with rainbow vertex colors): triangle, rectangle, pentagon, hexagon, circle, ellipse, trapezoid, star, arrow.

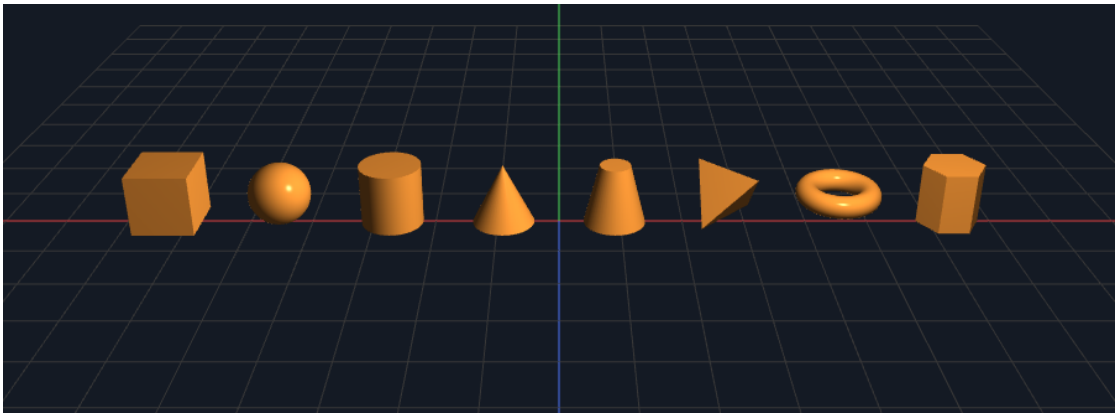
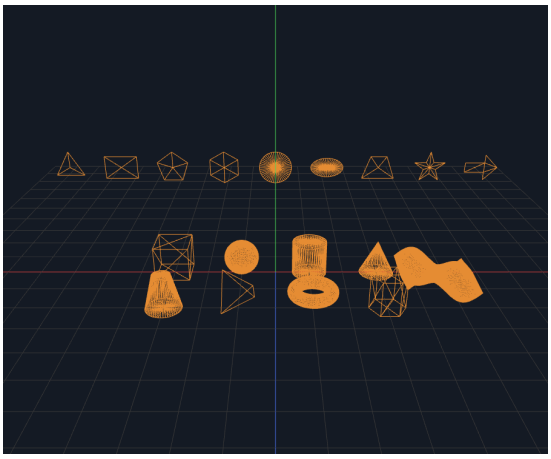
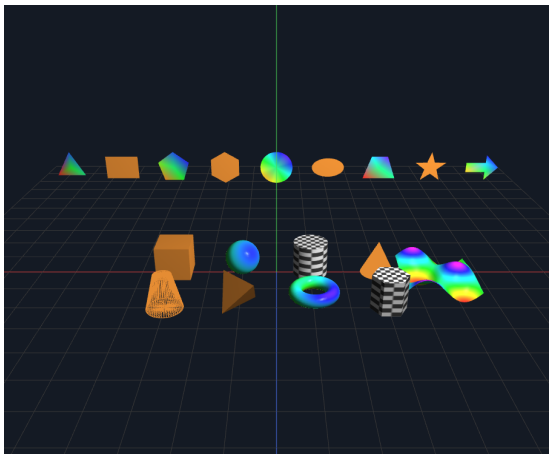


Figure 6: 3D solids (Phong): cube, sphere, cylinder, cone, truncated cone, tetrahedron, torus, prism.

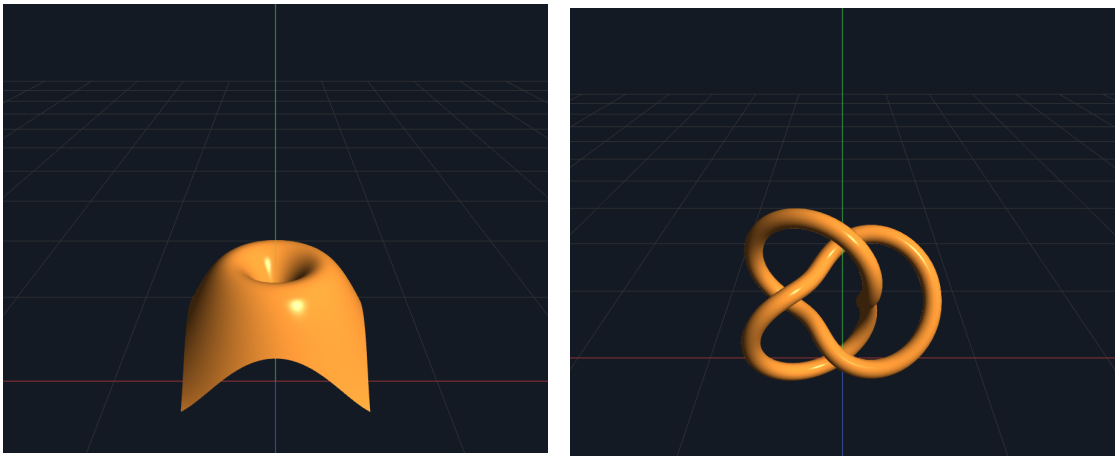


(a) All shapes in wireframe: the triangle meshes



(b) Demo scene: a mix of render modes

Figure 7: Wireframe and mixed-mode scenes.



(a) $z = 2\sin\sqrt{x^2+y^2}$, range $[-3,3]^2$ (b) Imported `torusknot.obj`

Figure 8: A mathematical surface and an imported model, both in Phong mode.

3.8 Limitations

ImGui has no native file dialog, so models and textures are chosen from the `assets/` folders or by typing a path; objects are selected from the list (there is no picking by mouse click); OBJ faces are fan-triangulated, which assumes convex faces; material files (`.mtl`) are ignored; Gouraud shading interpolates the lit vertex color, so highlights depend on mesh resolution (this is intended and shown in the comparison with Phong).

4 Part 2: Visualization of Atoms and Molecules

Part 2 asks for an interactive application that (i) visualizes atoms with Bohr's model, electrons being small spheres that revolve around the nucleus in distinct shells, (ii) visualizes simple molecules with the ball-and-stick model, atoms being colored spheres (red oxygen, white hydrogen, black carbon) and bonds cylinders, and optionally (iii) animates the electron motion and the vibration or rotation of the molecules. The grading scheme adds three concrete criteria: atoms modelled as spheres and simple molecules visualized; *configuration of atomic types and positions*; molecule structures rendered with proper camera control, with output screenshots.

The application (`part2_molecules`) is built around a scene graph of `SceneNode`s that share one sphere mesh, one cylinder mesh and one torus per Bohr shell, a single per-fragment Phong shader, and a Dear ImGui control panel with an element selector, a molecule selector, a molecule editor (element and position of every atom, bonds and bond orders, automatic bond detection), a geometry read-out, three animations and a screenshot function. Table 2 maps each requirement to the section that describes it. The mathematics behind every transformation is collected in Section 4.2, and the properties

claimed in the text (electrons on their rings, bonds joining their atoms, fixed centre of mass, …) are checked automatically by `make test` (Section 4.8).

Requirement	**Implementation (section)**
GUI with menus / toolbars to access and toggle the features	control panel: view mode, element list, molecule list, editor, animation toggles, lighting, camera, screenshot (4.7)
Bohr model: electrons as small spheres orbiting the nucleus in distinct shells	23 elements, one tilted orbit plane per shell, Z electron spheres (4.3)
Simple molecules: atoms as colored spheres, bonds as cylinders	8 molecules with experimental geometry, CPK colors, bonds of 1, 2 or 3 strands (4.4)
Configuration of atomic types and positions	element of the Bohr atom; molecule editor: element and position of each atom, add / remove atoms and bonds, bond orders, automatic bond detection (4.4.5)
Proper camera control, output screenshots	orbit / pan / zoom camera with automatic framing (4.2.4); `F12`, `Shift+F12` and `--screenshot` (4.7)
Optional: electron motion; molecular vibration or rotation	orbit animation (4.3.4); symmetric stretching about the centre of mass; rotation about the vertical axis through the centre of mass (4.5)
Suggestions: scene graph, transformation matrices, materials and lighting	4.1, 4.2, 4.6

Table 2: Part 2 requirements and where they are met.

4.1 Architecture and Components

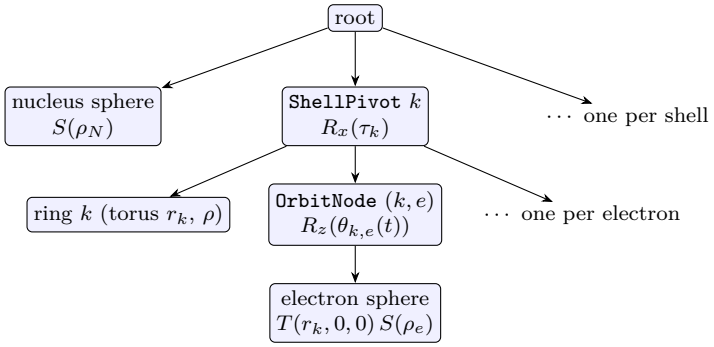
4.1.1 Modules

File(s)	Responsibility
`main.cpp`	Window (GLFW), callbacks (mouse, keyboard, resize), render loop, ImGui control panel (element list, molecule list, molecule editor, geometry read-out), automatic camera framing, screenshots, command-line options.
`Mesh.h / .cpp`	Vertex format (position + normal, 24 bytes); CPU generators `sphereData`, `cylinderData`, `torusData` (also used by the tests) and their upload to a VAO / VBO / EBO; `draw()`.
`SceneNode.h / .cpp`	Scene-graph node: local transform (TRS), `Material`, non-owning pointer to a shared `Mesh`, children owned by `std::unique_ptr`; recursive `draw()` (world = parent × local), `updateTree()`, `setWireframe()`.
`AtomData.h / .cpp`	Table of 23 elements (symbol, name, Z, standard atomic weight, covalent radius, CPK color, shell occupation); 8 `MoleculeSpecs` (experimental geometry in Å, bonds with their order, wavenumber of the symmetric stretch); distance-based bond detection; Hill formula.
`BohrModel.h / .cpp`	`BohrAtom` builder; `ShellPivot` (tilt of a shell) and `OrbitNode` (rotating pivot of one electron); per-shell radii, tilts and speeds.
`MoleculeScene.h / .cpp`	Ball-and-stick builder (centre of mass, similarity scaling); `VibrationNode` (symmetric stretch), `MolRotNode` (rotation); bond cylinders re-placed every frame.
`Screenshot.h / .cpp`	Reads the back buffer (`glReadPixels`) and writes a PNG with `stb_image_write` [8].
`phong.vert / .frag`	GLSL 330: transformation of positions and normals; per-fragment Phong lighting.
`tests/test_part2.cpp`	CPU checks of the geometry, the scene graphs and the animations (`make test`).
`common/Shader, Camera`	Shared shader loader and orbit camera (Section 2).

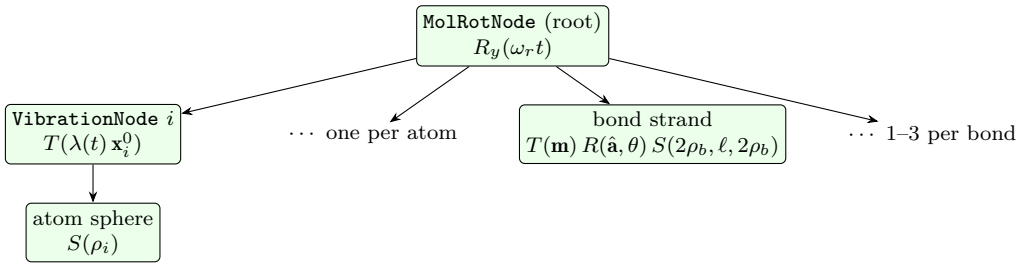
4.1.2 Scene graph design

The central design pattern is a **hierarchical scene graph**. Each `SceneNode` stores a local transform made of a position, a rotation (axis and angle) and a scale, an optional pointer to a shared mesh, a material and its children. Drawing is recursive: a node computes $M_{\text{world}} = M_{\text{parent}}\, M_{\text{local}}$, uploads the model and normal matrices and its material, issues the draw call and recurses into its children with M_{world} (Section 4.2.2). Moving a node therefore moves its whole subtree; this is what attaches an electron to its rotating orbit pivot and the atoms and bonds of a molecule to the rotating root. Animations are expressed as `update(dt)` overrides of three node types (`OrbitNode`, `VibrationNode`,

`MolRotNode`) and are applied by one recursive `updateTree()` before drawing. Figure 9 shows both trees.



(a) Bohr atom (`BohrAtom`)



(b) Ball-and-stick molecule (`MoleculeScene`)

Figure 9: Scene graphs of Part 2; each box shows the local transform of the node (notation of Sections 4.3.4, 4.4.2 and 4.4.3).

A Bohr atom with S shells and Z electrons has $N_{\text{nodes}} = 2 + 2S + 2Z$ nodes (root, nucleus, one pivot and one ring per shell, one orbit pivot and one sphere per electron): 18 for carbon, 70 for zinc. A molecule with n atoms and B_1, B_2, B_3 single, double and triple bonds has $1 + 2n + B_1 + 2B_2 + 3B_3$ nodes. All spheres of the application share one mesh, all bonds one cylinder, and ring k of every atom uses torus k; only the uniforms differ between nodes.

4.1.3 Data flow and frame loop

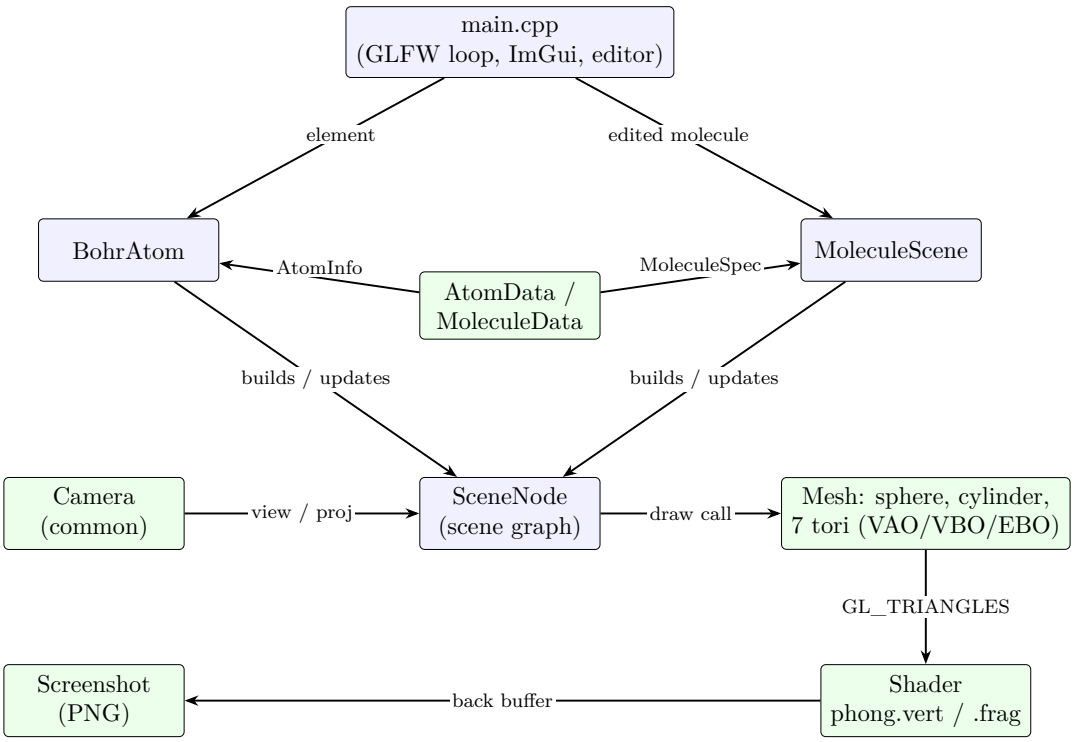


Figure 10: Part 2 data flow. `main` selects either a `BohrAtom` or a `MoleculeScene` (built from the molecule being edited); both read `AtomData` and build a scene graph whose nodes share the GPU meshes.

Each frame, `main` (i) polls GLFW events, (ii) computes Δt, (iii) rebuilds the active scene if the element or the molecule changed, (iv) lets the scene update its animations, (v) clears the framebuffer and sets the viewport to the right of the control panel, (vi) uploads the view, projection and light uniforms, (vii) draws the scene graph, (viii) saves a screenshot of the 3D view if one was requested, (ix) draws the ImGui panel (and saves a full-window screenshot if requested) and (x) swaps the buffers.

4.2 Mathematical Foundations

This section collects the transformations used by Part 2. Points $\mathbf{p} \in \mathbb{R}^3$ are written in homogeneous coordinates $(\mathbf{p}, 1)$ and directions as $(\mathbf{v}, 0)$, so that a translation acts on points but not on directions.

4.2.1 Local transform and axis–angle rotations

A node's local matrix is:

$$M_{\text{local}} = T(\mathbf{t})\, R(\hat{\mathbf{a}}, \theta)\, S(\mathbf{s}),$$
$$T(\mathbf{t}) = \begin{pmatrix} I_3 & \mathbf{t} \\ \mathbf{0}^T & 1 \end{pmatrix}, \qquad R = \begin{pmatrix} R_3 & \mathbf{0} \\ \mathbf{0}^T & 1 \end{pmatrix}, \qquad S = \operatorname{diag}(s_x, s_y, s_z, 1),$$

so that $M_{\text{local}}\mathbf{p} = \mathbf{t} + R_3\,(\mathbf{s} \odot \mathbf{p})$: a vertex is scaled, then rotated, then translated. The rotation by angle θ about the unit axis $\hat{\mathbf{a}}$ is given by Rodrigues' formula:

$$R_3(\hat{\mathbf{a}}, \theta) = I + \sin\theta\, K + (1 - \cos\theta)\, K^2, \qquad K = [\hat{\mathbf{a}}]_\times = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}, \qquad (1)$$

or, applied to a vector, $R_3\mathbf{v} = \mathbf{v}\cos\theta + (\hat{\mathbf{a}} \times \mathbf{v})\sin\theta + \hat{\mathbf{a}}\,(\hat{\mathbf{a}} \cdot \mathbf{v})(1 - \cos\theta)$. The rotations about the coordinate axes used below are the special cases

$$R_x(\tau) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\tau & -\sin\tau \\ 0 & \sin\tau & \cos\tau \end{pmatrix}, \qquad R_y(\psi) = \begin{pmatrix} \cos\psi & 0 & \sin\psi \\ 0 & 1 & 0 \\ -\sin\psi & 0 & \cos\psi \end{pmatrix},$$
$$R_z(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Whith 2 facts are used: rotations are isometries ($\|R\mathbf{v}\| = \|\mathbf{v}\|$), and $R^{-1} = R^T$.

4.2.2 Composition along the hierarchy

For a node v whose path from the root is $v_0, v_1, \ldots, v_k = v$,

$$M_{\text{world}}(v) = M_{\text{local}}(v_0)\, M_{\text{local}}(v_1) \cdots M_{\text{local}}(v_k),$$

computed recursively as

$$M_{\text{world}}(v) = M_{\text{world}}(\text{parent}(v))\, M_{\text{local}}(v)$$

with $M_{\text{world}}(\text{root's parent}) = I$. Each node costs one 4×4 product (and one inversion for the normal matrix). Because matrix products are associative, changing M_{local} of one node changes the world matrix of exactly its subtree, by the same left factor: the subtree moves rigidly with its root.

4.2.3 Transforming normals

Proposition 1 (normal matrix) *Let A be the upper-left 3×3 block of an invertible model matrix M. If $\mathbf{n}$ is normal to a surface at $\mathbf{p}$, then $\mathbf{n}' = A^{-T}\mathbf{n}$ is normal to the transformed surface at $M\mathbf{p}$.*

Proof. A tangent vector is the limit of a difference of two surface points; the translation cancels in the difference, so tangents transform as $\mathbf{t}' = A\mathbf{t}$. Then $\mathbf{n}'^T\mathbf{t}' = (A^{-T}\mathbf{n})^T A\mathbf{t} = \mathbf{n}^T A^{-1}A\,\mathbf{t} = \mathbf{n}^T\mathbf{t} = 0$ for every tangent $\mathbf{t}$. □.

The normal matrix is $(M^{-1}_{\text{world}})^T$ restricted to 3×3; for an affine $M = \left(\begin{smallmatrix} A & \mathbf{t} \\ 0 & 1 \end{smallmatrix}\right)$: this block is exactly A^{-T}, since $M^{-1} = \left(\begin{smallmatrix} A^{-1} & -A^{-1}\mathbf{t} \\ 0 & 1 \end{smallmatrix}\right)$. For a rotation $A^{-T} = A$, and for a uniform scale A^{-T} is a multiple of A. The bond cylinders are scaled non-uniformly, $A = R_3\,\mathrm{diag}(2\rho, \ell, 2\rho)$; their normals ($\pm\hat{\mathbf{y}}$ on the caps, $(\cos\varphi, 0, \sin\varphi)$ on the side) happen to be eigenvectors of the scale, so $A\mathbf{n}$ and $A^{-T}\mathbf{n}$ point the same way.

4.2.4 Camera, view and projection

Eye position. The shared orbit camera (Section 2.3.2) places the eye at $\mathbf{e} = \mathbf{c} + d\,(\cos\varphi\cos\psi,\ \sin\varphi,\ \cos\varphi\sin\psi)$ around the target $\mathbf{c}$, with yaw ψ, pitch φ and distance d. Our Part 2 resets it to $\psi = 90°$, $\varphi = 30°$, for example, $\mathbf{e} = d\,(0, \frac{1}{2}, \frac{\sqrt{3}}{2})$: the camera is on the $+Z$ side, so the world X axis points to the right, Y up and Z towards the viewer, and the default light $(5, 8, 5)$ is in front of the scene, above and to the right.

View matrix. `lookAt` builds the orthonormal camera basis

$$\mathbf{f} = \frac{\mathbf{c} - \mathbf{e}}{\|\mathbf{c} - \mathbf{e}\|}, \qquad \mathbf{r} = \frac{\mathbf{f} \times \hat{\mathbf{y}}}{\|\mathbf{f} \times \hat{\mathbf{y}}\|}, \qquad \mathbf{u} = \mathbf{r} \times \mathbf{f}, \qquad V = \begin{pmatrix} \mathbf{r}^T & -\mathbf{r}\cdot\mathbf{e} \\ \mathbf{u}^T & -\mathbf{u}\cdot\mathbf{e} \\ -\mathbf{f}^T & \mathbf{f}\cdot\mathbf{e} \\ \mathbf{0}^T & 1 \end{pmatrix},$$

which maps the eye to the origin and the viewing direction to $-Z$. The basis is undefined when $\mathbf{f} \parallel \hat{\mathbf{y}}$ ($\mathbf{f} \times \hat{\mathbf{y}} = \mathbf{0}$); this is why the pitch is clamped to $[-89°, 89°]$.

Projection and viewport. With vertical field of view α (45°), aspect ratio a, near $n = 0.1$ and far $f = 100$,

$$P = \begin{pmatrix} \frac{1}{a\tan(\alpha/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\alpha/2)} & 0 & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{pmatrix},$$

$$\mathbf{p}_{\text{clip}} = P\,V\,M_{\text{world}}\,(\mathbf{p}, 1)^T, \qquad \mathbf{p}_{\text{ndc}} = \mathbf{p}_{\text{clip}}/w_{\text{clip}}, \qquad w_{\text{clip}} = -z_{\text{eye}}.$$

The depth $z_{\text{ndc}} = \frac{f+n}{f-n} + \frac{2fn}{(f-n)\,z_{\text{eye}}}$ is hyperbolic in z_{eye}, so the depth precision is concentrated near the near plane. Our control panel covers the left 330 px of the window, so the scene is drawn in the viewport $x \in [330, W)$ with $a = (W - 330)/H$; using the aspect of the viewport (not of the window) keeps circles circular.

Automatic framing.

Proposition 2 *A sphere of radius R centred on the viewing axis at distance d lies inside the view cone of half-angle $\alpha/2$ if and only if $d \sin(\alpha/2) \geq R$.*

Proof. The distance from the centre of the sphere to a generator line of the cone is the leg opposite to the angle $\alpha/2$ in the right triangle whose hypotenuse is the segment of length d from the eye to the centre, i.e. $d \sin(\alpha/2)$. The sphere is inside the cone iff this distance is at least R. □

Since $a \geq 1$, the vertical half-angle is the binding one. When a scene is built or the view is reset, the camera distance is set to $d = 1.25\,R/\sin(\alpha/2)$ (25 % margin), with $R = r_{\text{outer}} + \rho_e$ for an atom (outermost shell plus electron radius) and $R = R_{\text{fit}} = 2.2$ for a molecule (as in Section 4.4.2): $d = 4.25$ (H), 7.19 (C), 10.78 (Na), 15.03 (Ca) and 7.19 for every molecule.

4.2.5 Phong lighting per fragment

Every fragment is lit with the Phong model of Section 2.3.3, with the light and material colors of the node:

$$C = L_a \odot m_a + L_d \odot m_d \max(N \cdot L, 0) + [N \cdot L > 0]\, L_s \odot m_s \max(R \cdot V, 0)^n, \qquad (2)$$

with $R = 2(N \cdot L)N - L$ (by GLSL `reflect(-L, N)`) and $[\,\cdot\,]$ equal to 1 if the condition holds and 0 otherwise.

(1) The normal interpolated across a triangle is shorter than one: for unit $\mathbf{a}, \mathbf{b}$:

$$\|(1-t)\mathbf{a} + t\mathbf{b}\|^2 = 1 - 2t(1-t)(1 - \mathbf{a} \cdot \mathbf{b}) < 1$$

so the fragment shader renormalises it.

(2) The indicator $[N \cdot L > 0]$: when the light is behind the surface, $R \cdot N = N \cdot L < 0$ so R points into the surface, yet $R \cdot V$ can still be positive at grazing viewing angles, which produced highlights on the dark side, the specular term is now evaluated only where the diffuse term is non-zero.

4.3 Visualizing Atoms: Bohr's Model

4.3.1 Physical model and brief physical mechanism

In Bohr's model (1913) [9, 10] an electron of mass m_e moves on a circle of radius r around a nucleus of charge Ze, the Coulomb attraction providing the centripetal force, and its angular momentum is quantised:

$$\frac{Ze^2}{4\pi\varepsilon_0 r^2} = \frac{m_e v^2}{r}, \qquad L = m_e v r = n\hbar, \quad n = 1, 2, 3, \ldots$$

Substituting $v = n\hbar/(m_e r)$ into the force balance gives $Ze^2/(4\pi\varepsilon_0) = n^2\hbar^2/(m_e r)$, hence

$$\begin{aligned}
r_n &= \frac{n^2}{Z}\, a_0, & a_0 &= \frac{4\pi\varepsilon_0\hbar^2}{m_e e^2} \approx 0.529 \text{ Å}, \\
v_n &= \frac{Z}{n}\,\alpha c, & \alpha &= \frac{e^2}{4\pi\varepsilon_0\hbar c} \approx \frac{1}{137}, \\
E_n &= \tfrac{1}{2} m_e v_n^2 - \frac{Ze^2}{4\pi\varepsilon_0 r_n} = -\frac{Z^2}{n^2}\,\frac{\alpha^2 m_e c^2}{2} & &\approx -13.6 \text{ eV} \cdot \frac{Z^2}{n^2}, \\
\omega_n &= \frac{v_n}{r_n} = \frac{Z^2}{n^3}\,\frac{\alpha c}{a_0}, & T_n &= \frac{2\pi}{\omega_n} = \frac{n^3}{Z^2}\, T_1, \quad T_1 \approx 1.52 \times 10^{-16} \text{ s.}
\end{aligned} \tag{3}$$

The orbits are concentric circles whose radius grow like n^2, and the outer electrons are slower ($v \propto 1/n$) and turn much more slowly ($\omega \propto 1/n^3$).

Shells and their occupation. Shell n (K, L, M, N for $n = 1..4$) can hold at most $2n^2$ electrons: in the quantum-mechanical refinement of the model a shell contains the subshells $l = 0, \ldots, n-1$, each with $2l+1$ orbitals of two spin states, and $\sum_{l=0}^{n-1} 2(2l+1) = 2n^2$ (Pauli principle). The table of `AtomData.cpp` stores, for each element, the number of electrons in each shell, for instance, the electrons of all subshells with the same n (Table 3). Shells do not always fill in order: potassium and calcium place electrons in $n = 4$ while the M shell holds only 8 of its 18 electrons, because the $4s$ subshell lies below $3d$ (Madelung $n + l$ rule). For example: iron is $[\mathrm{Ar}]\, 3d^6 4s^2 \to (2, 8, 14, 2)$ and copper $[\mathrm{Ar}]\, 3d^{10} 4s^1 \to (2, 8, 18, 1)$. The tests check $\sum_k n_k = Z$ and $n_k \le 2k^2$ for every element.

El.	Z	shells	El.	Z	shells	El.	Z	shells
H	1	1	Ne	10	2, 8	Cl	17	2, 8, 7
He	2	2	Na	11	2, 8, 1	Ar	18	2, 8, 8
Li	3	2, 1	Mg	12	2, 8, 2	K	19	2, 8, 8, 1
Be	4	2, 2	Al	13	2, 8, 3	Ca	20	2, 8, 8, 2
B	5	2, 3	Si	14	2, 8, 4	Fe	26	2, 8, 14, 2
C	6	2, 4	P	15	2, 8, 5	Cu	29	2, 8, 18, 1
N	7	2, 5	S	16	2, 8, 6	Zn	30	2, 8, 18, 2
O	8	2, 6						
F	9	2, 7						
capacity $2n^2$: K 2, L 8, M 18, N 32								

Table 3: The 23 elements and their shell occupation (electrons in K, L, M, N).

4.3.2 Mitigating from the physical model to the visual model

Equations (3) cannot be drawn literally. With $r_n \propto n^2$ the N shell is 16 times larger than the K shell: at the zoom where the N shell fits in the window the K shell would be a few pixels wide. With $\omega_n \propto n^{-3}$ an N electron needs 64 times longer than a K electron for one revolution . And the real period, $T_1 \approx 1.5 \times 10^{-16}$ s, must in any case be slowed down by about 2.6×10^{16}. The visual model keeps the qualitative laws (larger and slower outer shells) but compresses their range:

$$r_k = 0.1\,k^2 + 0.6\,k + 0.5 \quad (k = 1, \ldots, 4): \qquad r_k = 1.2,\ 2.1,\ 3.2,\ 4.5,$$

still a quadratic in k, like Bohr's, but dominated by its linear term ($r_4/r_1 = 3.75$ instead of 16), with a spacing $r_{k+1} - r_k \geq 0.9$ much larger than the electron diameter $2\rho_e = 0.2$. The angular speeds $\omega_k = 90, 60, 45, 35°/\text{s}$ decrease with k, but the linear speeds $v_k = \omega_k r_k$ are almost equal, so that every electron visibly moves (Table 4).

Shell k	K (1)	L (2)	M (3)	N (4)	O (5)	P (6)	Q (7)
Capacity $2k^2$	2	8	18	32	50	72	98
Radius r_k (world units)	1.2	2.1	3.2	4.5	5.9	7.4	9.0
Bohr ratio $r_k/r_1 = k^2$	1	4	9	16	25	36	49
Visual ratio r_k/r_1	1	1.75	2.67	3.75	4.92	6.17	7.50
Tilt τ_k (deg)	0	25	40	15	45	70	30
Angular speed ω_k (deg/s)	90	60	45	35	28	22	18
Linear speed $v_k = \omega_k r_k$ (units/s)	1.88	2.20	2.51	2.75	2.88	2.84	2.83
Apparent ring ratio $\lvert\cos(\tau_k + 30°)\rvert$	0.87	0.57	0.34	0.71	0.26	0.17	0.50

Table 4: Layout of the shells (as in `BohrModel.cpp`), the last row is derived in Section 4.3.4. Other visual constants: nucleus radius $\rho_N = 0.45$, electron radius $\rho_e = 0.1$, ring tube radius $\rho = 0.03$.

4.3.3 Scene graph of an atom

When an element is selected, `BohrAtom::build()` reads its `AtomInfo` and builds the tree of Figure 9a:

Node	**Purpose**
`root` (SceneNode)	identity transform, groups everything
`nucleus`	shared sphere, scale $\rho_N = 0.45$, diffuse = CPK color of the element, shininess 128
`ShellPivot`$_k$	static rotation $R_x(\tau_k)$: tilts the plane of shell k
`ring`$_k$	torus k (major radius r_k, tube $\rho = 0.03$), identity transform, gray-blue diffuse
`OrbitNode`$_{k,e}$	rotation $R_z(\theta_{k,e}(t))$ about the local Z axis (normal of the ring plane), animated
`electron`$_{k,e}$	shared sphere, $T(r_k, 0, 0)\, S(\rho_e)$, yellow diffuse

Because the ring and the orbit pivots of a shell are children of the same `ShellPivot`, they share its tilted frame; inside that frame the ring lies in the XY plane and the orbit pivots rotate about Z, which is why the electrons travel exactly along the ring (Proposition 3).

4.3.4 Electron kinematics

Composing the local matrices from the root to electron e ($e = 0, \dots, n_k - 1$) of shell k gives the centre of the electron sphere:

$$\mathbf{c}_{k,e}(t) = R_x(\tau_k)\, R_z(\theta)\, T(r_k, 0, 0)\, \mathbf{0} = r_k \begin{pmatrix} \cos\theta \\ \cos\tau_k \sin\theta \\ \sin\tau_k \sin\theta \end{pmatrix},$$

$$\theta = \theta_{k,e}(t) = \frac{360^\circ e}{n_k} + s\,\omega_k t \tag{4}$$

(angles taken modulo 360°), where s is the speed multiplier of the GUI (indeed $R_z(\theta)(r,0,0)^T = r(\cos\theta, \sin\theta, 0)^T$ and $R_x(\tau)(x,y,0)^T = (x,\, y\cos\tau,\, y\sin\tau)^T$). The initial phases $360^\circ e/n_k$ spread the electrons of a shell evenly.

Proposition 3 (electrons stay on their ring) *For every t, $\|\mathbf{c}_{k,e}(t)\| = r_k$ and $\mathbf{c}_{k,e}(t) \cdot \mathbf{n}_k = 0$ with $\mathbf{n}_k = R_x(\tau_k)\hat{\mathbf{z}} = (0, -\sin\tau_k, \cos\tau_k)$. Hence every electron of shell k moves on the circle $\{\mathbf{x} : \|\mathbf{x}\| = r_k,\ \mathbf{x} \cdot \mathbf{n}_k = 0\}$, which is the centre circle of ring k.*

Proof. R_x and R_z are isometries, so $\|\mathbf{c}\| = \|(r_k, 0, 0)\| = r_k$. From (4), $\mathbf{c} \cdot \mathbf{n}_k = r_k \sin\theta\,(-\cos\tau_k \sin\tau_k + \sin\tau_k \cos\tau_k) = 0$. The ring is the torus generated in the XY plane with centre circle $\{\|\mathbf{x}\| = r_k,\ x_3 = 0\}$, transformed by the same $R_x(\tau_k)$, which maps the plane $x_3 = 0$ to the plane $\mathbf{x} \cdot \mathbf{n}_k = 0$.

Proposition 4 (no overlaps) *For all elements of Table 3 and all t: (a) two electrons of different shells are at least 0.9 apart; (b) two electrons of shell k are at least $2r_k \sin(\pi/n_k) \geq 1.11$ apart; (c) every electron is at distance $r_k \geq 1.2$ from the centre. Since $2\rho_e = 0.2$ and $\rho_N + \rho_e = 0.55$, electron spheres never intersect each other or the nucleus.*

Proof. (a) By the reverse triangle inequality and Proposition 3, $\|\mathbf{c} - \mathbf{c}'\| \geq \big|\|\mathbf{c}\| - \|\mathbf{c}'\|\big| = |r_k - r_{k'}| \geq 0.9$.
(b) All electrons of a shell share ω_k, so their phase differences stay equal to $360^\circ m/n_k$, $m \neq 0$; two points of a circle of radius r separated by the angle Δ are at the chord distance $2r\sin(\Delta/2) \geq 2r\sin(\pi/n_k)$. The smallest value over the 23 elements is the full M shell of Cu and Zn: $2 \cdot 3.2 \cdot \sin 10^\circ = 1.11$ (L shell, $n = 8$: 1.61; M shell of Fe, $n = 14$: 1.42). (c) follows from Proposition 3. □

Time integration. Each `OrbitNode` adds $s\,\omega_k\,\Delta t$ to its angle every frame, with Δt measured by `glfwGetTime`. For a constant angular speed this explicit update is exact (the increments sum to $s\,\omega_k t$), so the motion does not depend on the frame rate. Taking the angle modulo 360° keeps it in $[0, 360)$, where a 32-bit float has a resolution of about

3×10^{-5} degrees; without the modulo the angle would reach 3.2×10^{5} degrees after one hour, with a resolution of only $0.03°$. Moving the *Speed* slider changes ω of all orbit nodes in place, so the electrons keep their phases. The rotation axis of an orbit node is set when it is created, so an atom built while the animation is paused is drawn correctly too.

Apparent shape of the rings. At these distances the projection is close to orthographic, so ring k appears as an ellipse whose axis ratio is the cosine of the angle between its normal and the viewing direction, $b/a = |\mathbf{n}_k \cdot \hat{\mathbf{v}}|$. In the default view $\hat{\mathbf{v}} = -(0, \frac{1}{2}, \frac{\sqrt{3}}{2})$, so $b/a = \left|\frac{1}{2}\sin\tau_k - \frac{\sqrt{3}}{2}\cos\tau_k\right| = |\cos(\tau_k + 30°)|$: 0.87, 0.57, 0.34 and 0.71 for K, L, M and N (Table 4). The four rings appear as clearly different ellipses, none of them edge-on (which would hide the electrons of a full shell behind each other), and this gives the atom its three-dimensional look even in a still image. (The tilt of the M shell was lowered from $55°$ to $40°$ for this reason: $|\cos 85°| = 0.09$.)

4.3.5 Orbital rings: torus geometry

Ring k is the torus with major radius $R = r_k$ and tube radius ρ,

$$\mathbf{p}(\vartheta, \phi) = \big((R + \rho\cos\phi)\cos\vartheta,\ (R + \rho\cos\phi)\sin\vartheta,\ \rho\sin\phi\big),$$
$$\mathbf{n}(\vartheta, \phi) = (\cos\phi\cos\vartheta,\ \cos\phi\sin\vartheta,\ \sin\phi) = \frac{\mathbf{p} - R(\cos\vartheta, \sin\vartheta, 0)}{\rho},$$

sampled at $\vartheta_i = 2\pi i/M$, $\phi_j = 2\pi j/m$ ($i = 0..M$, $j = 0..m$, vertex index $i(m+1) + j$). The quad between $a = (i, j)$ and $b = (i{+}1, j)$ is split into $(a, b, a{+}1)$ and $(b, b{+}1, a{+}1)$. Since $\partial_\vartheta\mathbf{p} \times \partial_\phi\mathbf{p} = \rho\,(R + \rho\cos\phi)\,\mathbf{n}$ with $R + \rho\cos\phi > 0$, these triangles are counter-clockwise when seen from outside, as required by back-face culling. $V = (M{+}1)(m{+}1)$, $I = 6Mm$; with $M = 128$, $m = 8$: $V = 1161$, $I = 6144$.

Lemma 5 *A uniform scaling by σ maps the torus (R, ρ) to the torus $(\sigma R, \sigma\rho)$.*

This is immediate from the parametrisation, and it is the reason for having one torus per shell. Scaling a single unit torus to the radius of each shell, as the previous version did with $\rho = 0.025$, gives tubes of radius $0.025\,r_k = 0.030,\ 0.053,\ 0.080,\ 0.1125$: the N ring was thicker than an electron ($\rho_e = 0.1$), whose sphere, centred on the centre circle, was then entirely inside the tube and invisible (K, Ca, Fe, Cu, Zn), and the M electrons barely showed. Now seven tori with $R = r_k$ and the same $\rho = 0.03 < \rho_e$ are built once at start-up, and the ring nodes are not scaled (the tests check $\det A = 1$ for every ring).

Polygonal error. The M-gon deviates from the true circle by at most the sagitta $\delta = r\big(1 - \cos(\pi/M)\big) \approx \pi^2 r/(2M^2)$: 3.6×10^{-4} for the K ring and 1.4×10^{-3} for the N ring, much smaller than ρ, so an electron (which moves on the exact circle) always appears centred in its ring.

4.3.6 Example: carbon

Carbon ($Z = 6$, shells $(2, 4)$) has 18 nodes. At $t = 0$ and with $s = 1$, Equation (4) gives:

Shell	r_k	τ_k	ω_k	electron centres at $t = 0$	$\rightarrow$	electron $e = 0$ at $t = 1$ s
K	1.2	0°	90°/s	$(\pm 1.2, 0, 0)$	$\rightarrow$	$(0, 1.2, 0)$
L	2.1	25°	60°/s	$(\pm 2.1, 0, 0)$, $\pm(0, 1.903, 0.888)$	$\rightarrow$	$(1.050, 1.648, 0.769)$

4.4 Visualizing Molecules: Ball-and-Stick Model

4.4.1 Molecule data

Eight molecules are predefined in `AtomData.cpp` for the ball-and-stick model [11], with experimental equilibrium geometries and the experimental wavenumber of their symmetric stretching vibration (used in Section 4.5). Each `MoleculeSpec` stores the atoms (element symbol and position in Å) and the bonds (two atom indices and the bond order).

Molecule	**Shape**	**Group**	**Bonds**	d (Å)	**Angle**	$\tilde{\nu}_1$ (cm^{-1})	ν_1 **active in**
H_2O	bent	C_{2v}	2 single	0.957	104.5°	3657	IR, Raman
CO_2	linear	$D_{\infty h}$	2 double	1.160	180°	1333	Raman
NH_3	trigonal pyramidal	C_{3v}	3 single	1.012	106.7°	3337	IR, Raman
CH_4	tetrahedral	T_d	4 single	1.087	109.47°	2917	Raman
O_2	linear	$D_{\infty h}$	1 double	1.2075	—	1556	Raman
N_2	linear	$D_{\infty h}$	1 triple	1.0977	—	2330	Raman
HCl	linear	$C_{\infty v}$	1 single	1.2746	—	2886	IR, Raman
NaCl	linear (ion pair)	$C_{\infty v}$	1 (drawn)	2.361	—	366	IR, Raman

Table 5: Predefined molecules: bond length d, bond angle and the symmetric stretch ν_1 (gas phase). NaCl is ionic; its "bond" is drawn to show the ion pair.

The coordinates follow from d and the bond angle ϑ in closed form (central atom at the origin):

- **Bent AX_2** (H_2O, in the XY plane): $\mathbf{X}_{1,2} = \left(\pm d \sin\frac{\vartheta}{2},\ d\cos\frac{\vartheta}{2},\ 0\right) = (\pm 0.757,\ 0.586,\ 0)$.
- **Pyramidal AX_3** (NH_3, threefold axis Y): $\mathbf{X}_j = (\varrho\cos 120°j,\ -h,\ \varrho\sin 120°j)$, $j = 0, 1, 2$. The three H atoms form an equilateral triangle of side $\varrho\sqrt{3}$, and the law of cosines in the triangle X_iAX_j gives $3\varrho^2 = 2d^2(1 - \cos\vartheta)$, so $\varrho = d\sqrt{\frac{2}{3}(1 - \cos\vartheta)} = 0.9373$ and $h = \sqrt{d^2 - \varrho^2} = 0.3815$ for $d = 1.012$, $\vartheta = 106.67°$. (The ammonia

data of the previous version was not a pyramid: its N–H lengths were 1.049, 1.049 and 0.933 Å and one H–N–H angle was 127°.)

- **Tetrahedral $\mathbf{AX}_4$** (CH_4): $\mathbf{X} = a(\pm1, \pm1, \pm1)$ with an even number of minus signs, $a = d/\sqrt{3} = 0.628$; $\cos\vartheta = \mathbf{X}_i \cdot \mathbf{X}_j / d^2 = -a^2/(3a^2) = -\frac{1}{3}$, i.e. $\vartheta = \arccos(-\frac{1}{3}) = 109.47°$.
- **Linear**: $(\pm d, 0, 0)$ around the central atom (CO_2) or $(\pm d/2, 0, 0)$ for a diatomic.

4.4.2 Normalisation: centre of mass and similarity scaling

Molecules have very different sizes (NaCl is 2.4 Å long, N_2 1.1 Å), so `MoleculeScene::build` maps every molecule into the same sphere of radius $R_{\text{fit}} = 2.2$ world units. With the standard atomic weights m_i and covalent radii r_i:

$$\mathbf{c} = \frac{\sum_i m_i \mathbf{p}_i}{\sum_i m_i}, \qquad s = \frac{R_{\text{fit}}}{\max_i \big(\|\mathbf{p}_i - \mathbf{c}\| + \kappa r_i\big)}, \qquad \mathbf{x}_i^0 = s\,(\mathbf{p}_i - \mathbf{c}), \qquad \rho_i = s\,\kappa\, r_i, \quad (5)$$

with $\kappa = \frac{1}{2}$; every bond radius is multiplied by s as well.

Proposition 6 *The map $\Phi(\mathbf{p}) = s(\mathbf{p} - \mathbf{c})$ is a similarity: $\|\Phi(\mathbf{p}) - \Phi(\mathbf{q})\| = s\|\mathbf{p} - \mathbf{q}\|$ and all angles are preserved. Since the radii are scaled by the same s, the rendered model is an exact scaled copy of the model in Ångström, its centre of mass is the origin, and $\max_i(\|\mathbf{x}_i^0\| + \rho_i) = R_{fit}$: the farthest ball touches the fit sphere, which the camera framing keeps in view.*

Proof. $\Phi(\mathbf{p}) - \Phi(\mathbf{q}) = s(\mathbf{p} - \mathbf{q})$, a positive multiple, which preserves ratios of lengths and angles; $\sum_i m_i \mathbf{x}_i^0 = s\big(\sum_i m_i \mathbf{p}_i - \mathbf{c} \sum_i m_i\big) = \mathbf{0}$; the last identity is the definition of s. □

Centring on the centre of mass (rather than on an arbitrary atom) is what a free molecule does: it rotates and vibrates about its centre of mass (Section 4.5). For water the centre of mass is $(0, 0.0656, 0)$ Å, almost on the oxygen ($m_O/m_H \approx 16$), and $s = 2.049$: oxygen ball 0.748, hydrogen ball 0.318, O–H distance 1.961 world units. The scale factors of the eight molecules are 2.05 (H_2O), 1.44 (CO_2), 1.92 (NH_3), 1.77 (CH_4), 2.27 (O_2), 2.38 (N_2), 1.58 (HCl) and 1.00 (NaCl).

Why $\kappa = \frac{1}{2}$. Covalent radii are approximately additive, $d_{ij} \approx r_i + r_j$. Balls of radius κr then cover $\kappa(r_i + r_j) \approx \kappa\, d_{ij}$ of every bond, and the visible part of each stick is about $(1 - \kappa)\, d_{ij} = d_{ij}/2$: the balls never touch (as they would in a space-filling model, $\kappa \geq 1$) and the sticks do not dominate. The previous version scaled the positions but not the radii, so the ball / stick proportions changed from one molecule to the other.

4.4.3 Bond cylinders

Mesh. The shared cylinder has radius $\frac{1}{2}$, height 1 and axis Y. With $N = 32$ segments and $a_i = 2\pi i/N$ it has a side ring of $2(N+1)$ vertices (bottom b_i at index $2i$, top t_i at $2i+1$, normal $(\cos a_i, 0, \sin a_i)$), two cap centres and $2(N+1)$ cap-rim vertices with normals $\pm\hat{\mathbf{y}}$: $V = 4N + 6 = 134$, $T = 4N = 128$, $I = 12N = 384$. The side quads are split into (b_i, t_i, b_{i+1}) and (t_i, t_{i+1}, b_{i+1}), for which

$$(\mathbf{t}_i - \mathbf{b}_i) \times (\mathbf{b}_{i+1} - \mathbf{b}_i) = (0, h, 0) \times (\Delta x, 0, \Delta z) = h\,(\Delta z, 0, -\Delta x) \approx h\, r\, \Delta a\,(\cos a_i, 0, \sin a_i),$$

an outward vector: the triangles are counter-clockwise from outside (the caps likewise). The previous version used (b_i, b_{i+1}, t_i), whose normal points inwards; with back-face culling the near half of every bond was discarded and the far half was drawn with normals pointing away from the viewer, so the bonds were dull and had no highlight.

Placement. For a strand from $\mathbf{a}$ to $\mathbf{b}$ let $\ell = \|\mathbf{b} - \mathbf{a}\|$, $\hat{\mathbf{d}} = (\mathbf{b} - \mathbf{a})/\ell$, $\mathbf{m} = (\mathbf{a} + \mathbf{b})/2$ and

$$M_{\text{bond}} = T(\mathbf{m})\, R(\hat{\mathbf{a}}, \theta)\, S(2\rho_b,\ \ell,\ 2\rho_b), \qquad \hat{\mathbf{a}} = \frac{\hat{\mathbf{y}} \times \hat{\mathbf{d}}}{\|\hat{\mathbf{y}} \times \hat{\mathbf{d}}\|}, \qquad \theta = \operatorname{atan2}\big(\|\hat{\mathbf{y}} \times \hat{\mathbf{d}}\|,\ \hat{\mathbf{y}} \cdot \hat{\mathbf{d}}\big). \tag{6}$$

Proposition 7 *$M_{bond}(0, -\frac{1}{2}, 0) = \mathbf{a}$, $M_{bond}(0, \frac{1}{2}, 0) = \mathbf{b}$, and the transformed cylinder has radius ρ_b.*

Proof. $S(0, \pm\frac{1}{2}, 0) = (0, \pm\frac{\ell}{2}, 0)$. Write $\hat{\mathbf{d}} = \cos\theta\, \hat{\mathbf{y}} + \sin\theta\, \hat{\mathbf{w}}$ with $\hat{\mathbf{w}} \perp \hat{\mathbf{y}}$ a unit vector and $\theta \in [0, \pi]$; this is possible and consistent with (6) because $\hat{\mathbf{y}} \cdot \hat{\mathbf{d}} = \cos\theta$ and $\|\hat{\mathbf{y}} \times \hat{\mathbf{d}}\| = \sin\theta$. Then $\hat{\mathbf{y}} \times \hat{\mathbf{d}} = \sin\theta\,(\hat{\mathbf{y}} \times \hat{\mathbf{w}})$, so $\hat{\mathbf{a}} = \hat{\mathbf{y}} \times \hat{\mathbf{w}}$, which is perpendicular to $\hat{\mathbf{y}}$. Rodrigues' formula (1) with $\hat{\mathbf{a}} \cdot \hat{\mathbf{y}} = 0$ and $\hat{\mathbf{a}} \times \hat{\mathbf{y}} = (\hat{\mathbf{y}} \times \hat{\mathbf{w}}) \times \hat{\mathbf{y}} = \hat{\mathbf{w}}(\hat{\mathbf{y}} \cdot \hat{\mathbf{y}}) - \hat{\mathbf{y}}(\hat{\mathbf{w}} \cdot \hat{\mathbf{y}}) = \hat{\mathbf{w}}$ gives $R\hat{\mathbf{y}} = \cos\theta\, \hat{\mathbf{y}} + \sin\theta\, \hat{\mathbf{w}} = \hat{\mathbf{d}}$. Hence $M_{\text{bond}}(0, \pm\frac{1}{2}, 0) = \mathbf{m} \pm \frac{\ell}{2}\hat{\mathbf{d}}$, which are $\mathbf{b}$ and $\mathbf{a}$. The scale maps the circle of radius $\frac{1}{2}$ in the XZ plane to radius ρ_b, and R and T preserve distances. □

When $\hat{\mathbf{y}} \times \hat{\mathbf{d}} = \mathbf{0}$ the axis is undefined: $\hat{\mathbf{d}} = \hat{\mathbf{y}}$ needs no rotation and $\hat{\mathbf{d}} = -\hat{\mathbf{y}}$ needs a half turn about any axis perpendicular to $\hat{\mathbf{y}}$ (the code uses $\hat{\mathbf{x}}$). atan2 is preferred to $\arccos(\hat{\mathbf{y}} \cdot \hat{\mathbf{d}})$ because $\arccos'(x) = -1/\sqrt{1 - x^2}$ diverges at $x = \pm 1$: near-vertical bonds would turn a rounding error ε of the dot product into an angle error of order $\sqrt{2\varepsilon}$, whereas atan2 uses both the sine and the cosine and is well-conditioned for every direction.

Remark: the previous implementation. The previous version built R with `glm::rotate` and then recovered the axis–angle pair from the matrix, $\theta = \arccos\frac{\operatorname{tr} R - 1}{2}$ and $\hat{\mathbf{a}} = \frac{1}{2\sin\theta}(R_{32} - R_{23},\ R_{13} - R_{31},\ R_{21} - R_{12})$, which is correct because $R - R^T =$

$2\sin\theta\,[\hat{\mathbf{a}}]_\times$ by (1). But GLM stores matrices column-major (`m[col][row]`) and the entries were read as `m[row][col]`, which yields $-\hat{\mathbf{a}}$, i.e. the rotation $R^T = R^{-1}$. Since $R^T\hat{\mathbf{y}}$ is the mirror image $(-d_x, d_y, -d_z)$ of $\hat{\mathbf{d}}$, the bonds of H_2O, NH_3 and CH_4 crossed the molecule instead of joining the atoms (bonds along the X axis happened to look right, $(\mp1, 0, 0)$ and $(\pm1, 0, 0)$ being the same line). The current code stores $(\hat{\mathbf{a}}, \theta)$ of (6) directly, and Proposition 7 is checked for 300 random directions and all degenerate ones.

4.4.4 Multiple bonds

The strands of a double or triple bond are parallel copies of the bond shifted along

$$\hat{\mathbf{u}} = \frac{\hat{\mathbf{d}} \times \hat{\mathbf{z}}}{\|\hat{\mathbf{d}} \times \hat{\mathbf{z}}\|} \qquad \Big(\text{or } \frac{\hat{\mathbf{d}} \times \hat{\mathbf{x}}}{\|\hat{\mathbf{d}} \times \hat{\mathbf{x}}\|} \text{ if } \|\hat{\mathbf{d}} \times \hat{\mathbf{z}}\| < 0.1\Big),$$

a unit vector perpendicular to the bond and to Z: in the default view (camera on $+Z$) the strands lie side by side on the screen, as in a structural formula, and for the planar molecules of the XY plane they lie in the molecular plane. A double bond has two strands at $\pm\delta_2\hat{\mathbf{u}}$, a triple bond three coplanar strands at $0, \pm\delta_3\hat{\mathbf{u}}$, all of radius ρ_m (in Å, before scaling):

	radius	offset	conditions (all lengths multiplied by s)
single	$\rho_1 = 0.07$	0	inside the smallest ball: $\rho_1 < \kappa r_H = 0.155$
double	$\rho_m = 0.04$	$\pm\delta_2 = \pm0.07$	strands apart: $2\delta_2 = 0.14 > 2\rho_m = 0.08$
triple	$\rho_m = 0.04$	$0,\ \pm\delta_3 = \pm0.11$	strands apart: $\delta_3 = 0.11 > 2\rho_m$; ends hidden in the balls: $\delta_3 + \rho_m = 0.15 < \kappa r_O = 0.365$

The previous version shifted the strands along $\hat{\mathbf{d}} \times \hat{\mathbf{y}}$, which for bonds along X is $\pm Z$, the viewing direction: the two strands of CO_2 were drawn one behind the other and the third strand of N_2 was put on the perpendicular axis; a double bond along Y even produced NaN (a zero vector was normalised before the fallback test).

4.4.5 Configuring atom types and positions

Selecting a molecule copies its `MoleculeSpec` into a working copy which the editor of the control panel modifies:

- every atom has an element selector (the 23 elements) and a position field (x, y, z) in Å that can be dragged or typed, and a button to remove it (its bonds are removed and the other bonds renumbered); *Add atom* appends a hydrogen atom 1 Å beyond the atom with the largest x;
- every bond has an order slider (1–3) and a remove button; *Add bond* joins two atom indices (validated: in range, distinct, not already bonded);

- *Auto bonds* replaces the bond list by the result of the detection below; *Reset* restores the preset.

Every change marks the scene dirty and it is rebuilt in the next frame ($O(n+B)$ nodes), so the centre of mass, the scale (5) and the bond cylinders are recomputed; the formula shown becomes the Hill formula of the current atoms (C and H first, then alphabetical) followed by "(edited)", and the experimental wavenumber of the preset is discarded.

Automatic bond detection. Atoms $i < j$ are bonded if

$$\|\mathbf{p}_i - \mathbf{p}_j\| \le \tau\,(r_i + r_j), \qquad \tau = 1.2,$$

the additivity of covalent radii with a 20 % tolerance, tested for all $\binom{n}{2}$ pairs. For water, O–H: $0.957 \le 1.2(0.73 + 0.31) = 1.248$ (bonded) while H$\cdots$H: $1.514 > 1.2 \cdot 0.62 = 0.744$ (not bonded); for NaCl, $2.361 \le 3.036$. On the eight presets the detection finds exactly their bonds (tested). The bond order cannot be inferred from distances alone and is set by the user.

Geometry read-out. The panel lists, for the current atoms, every bond length $\ell_{ij} = \|\mathbf{p}_i - \mathbf{p}_j\|$ and every bond angle at an atom j shared by two bonds (i, j) and (j, k),

$$\vartheta_{ijk} = \arccos \frac{(\mathbf{p}_i - \mathbf{p}_j) \cdot (\mathbf{p}_k - \mathbf{p}_j)}{\|\mathbf{p}_i - \mathbf{p}_j\|\,\|\mathbf{p}_k - \mathbf{p}_j\|},$$

with the argument clamped to $[-1, 1]$ against rounding. For example, turning water into hydrogen sulfide (O→S and the H atoms moved to ($\pm$0.962, 0.927, 0)) shows S–H = 1.336 Å and H–S–H = 92.1°, the experimental values (Figure 15).

4.5 Animation (Optional)

The three animations are driven by Δt from the GLFW timer and live at different levels of the scene graph, so they compose without interfering.

4.5.1 Electron motion

Section 4.3.4: each `OrbitNode` turns its electron about the normal of the ring plane at $s\,\omega_k$; *Animate Electrons* pauses / resumes all of them and *Speed* sets $s \in [0.1, 5]$.

4.5.2 Molecular vibration: physical background

Near the equilibrium geometry $\mathbf{x}^0 \in \mathbb{R}^{3n}$ the potential energy is approximately quadratic, $U(\mathbf{x}^0 + \boldsymbol{\xi}) \approx U_0 + \frac{1}{2}\boldsymbol{\xi}^T H \boldsymbol{\xi}$ ($\nabla U = \mathbf{0}$ at equilibrium, H the Hessian), and Newton's law

gives $\mathsf{M}\ddot{\boldsymbol{\xi}} = -H\boldsymbol{\xi}$ with $\mathsf{M} = \mathrm{diag}(m_1 I_3, \ldots, m_n I_3)$. Solutions $\boldsymbol{\xi} = \mathbf{q}\cos(\omega t + \varphi)$ exist when

$$H\mathbf{q} = \omega^2 \mathsf{M}\mathbf{q} \quad \Longleftrightarrow \quad \big(\mathsf{M}^{-1/2} H \mathsf{M}^{-1/2}\big)\tilde{\mathbf{q}} = \omega^2 \tilde{\mathbf{q}}, \qquad \tilde{\mathbf{q}} = \mathsf{M}^{1/2}\mathbf{q},$$

a symmetric eigenvalue problem whose eigenvectors are the *normal modes* [12, 13]. Three translations and three rotations (two for a linear molecule) cost no energy and have $\omega = 0$, leaving $3n - 6$ vibrations ($3n - 5$ if linear): 3 for H_2O, 6 for NH_3, 9 for CH_4, 4 for CO_2 and 1 for a diatomic molecule. For a diatomic molecule the problem reduces to one oscillator in the bond length with the reduced mass $\mu = m_1 m_2/(m_1 + m_2)$:

$$\mu\,\ddot{x} = -k\,x, \qquad \omega = \sqrt{k/\mu}, \qquad \tilde{\nu} = \frac{\omega}{2\pi c}.$$

For HCl, $\mu = 0.980$ u and $\tilde{\nu} = 2886$ cm^{-1} give the force constant $k = \mu(2\pi c\tilde{\nu})^2 \approx 481$ N/m. A mode absorbs infrared light only if it changes the dipole moment ($\partial\boldsymbol{\mu}/\partial q \neq \mathbf{0}$) [14]: the stretch of the homonuclear O_2 and N_2 and the symmetric stretches of CO_2 and CH_4 are IR-inactive and observed in Raman spectra instead (Table 5).

4.5.3 Implemented vibration: symmetric stretching

Every atom is wrapped in a `VibrationNode` that moves it on the line through the centre of mass, all atoms in phase:

$$\mathbf{x}_i(t) = \lambda(t)\,\mathbf{x}_i^0, \qquad \lambda(t) = 1 + A\sin(2\pi f t), \tag{7}$$

with $\mathbf{x}_i^0$ from (5) (relative to the centre of mass), the relative amplitude A (default 0.08, slider 0–0.25) and the displayed frequency f.

Proposition 8 *For every molecule, the motion* (7) *(a) keeps the centre of mass at the origin, (b) has zero total linear momentum, (c) has zero total angular momentum, and (d) multiplies every interatomic distance by $\lambda(t)$ while keeping every bond direction and every bond angle constant.*

Proof. (a) $\sum_i m_i\mathbf{x}_i = \lambda\sum_i m_i\mathbf{x}_i^0 = \mathbf{0}$ by Proposition 6. (b) $\mathbf{P} = \sum_i m_i\dot{\mathbf{x}}_i = \dot{\lambda}\sum_i m_i\mathbf{x}_i^0 = \mathbf{0}$. (c) $\mathbf{L} = \sum_i m_i\,\mathbf{x}_i \times \dot{\mathbf{x}}_i = \lambda\dot{\lambda}\sum_i m_i\,\mathbf{x}_i^0 \times \mathbf{x}_i^0 = \mathbf{0}$. (d) $\mathbf{x}_i - \mathbf{x}_j = \lambda(\mathbf{x}_i^0 - \mathbf{x}_j^0)$ with $\lambda > 0$. □

So the motion is purely internal — no drift and no spurious rotation, properties shared by every normal mode. It is exactly the vibration of a diatomic molecule (the displacements are along the axis, with $\|\Delta\mathbf{x}_1\|/\|\Delta\mathbf{x}_2\| = m_2/m_1$: in HCl the hydrogen atom moves 35.2 times more than the chlorine atom), and exactly the symmetric stretch ν_1 of CO_2 (carbon at rest, the two oxygen atoms moving in opposite directions) and of CH_4 (carbon at rest, the four H atoms moving radially): these modes are the only ones of their symmetry species (Σ_g^+, A_1), so symmetry alone fixes their pattern. Water and ammonia have two totally symmetric modes (stretch and bend), and their true ν_1 contains a little bending; (7) is its stretching part. Asymmetric and bending modes would require the eigen-decomposition above and are not implemented.

Frequency. The displayed frequency is proportional to the experimental wavenumber, $f = 10^{-3}\tilde{\nu}_1$ Hz (times the *Vib. speed* multiplier): 3.66 Hz for water, 2.89 Hz for HCl, 1.33 Hz for CO_2, 0.37 Hz for NaCl. Relative frequencies are therefore physical (light atoms and stiff bonds vibrate faster, $\omega = \sqrt{k/\mu}$); the real motion, at $c\tilde{\nu}$, is slowed down by $c\tilde{\nu}/f = 3.0 \times 10^{13}$. An edited molecule, whose wavenumber is unknown, vibrates at 2 Hz.

Bonds follow the atoms. After the atoms have moved, every strand is placed again with (6) between the current $\mathbf{x}_a$ and $\mathbf{x}_b$ (plus its sideways offset), so the sticks stretch and shrink with the vibration. The previous version moved each atom independently along x, y or z at different frequencies and left the bonds in place: the centre of mass moved, the molecule rotated and the atoms drifted away from their bonds.

4.5.4 Molecular rotation

The root `MolRotNode` applies $R_y(\omega_r t)$ with $\omega_r = 30°/\text{s}$ about the vertical axis through the centre of mass (the origin after normalisation), the axis about which a free molecule rotates. Atom i is drawn with

$$M_{\text{world},i}(t) = R_y(\omega_r t)\; T\big(\lambda(t)\,\mathbf{x}_i^0\big)\; S(\rho_i),$$

and since $R_y(\lambda\mathbf{x}) = \lambda R_y\mathbf{x}$ (a rotation is linear), rotation and vibration commute: either can be switched on or off at any time. Bond strands are children of the same root, so they rotate with the atoms. Editing a molecule keeps its current rotation angle.

4.6 Rendering and Lighting

Vertex format and meshes. Each vertex has two attributes (24 bytes): `aPos` (location 0) and `aNormal` (location 1), both `vec3` in object space; no texture coordinates or vertex colors are needed because every node has a uniform material.

Mesh	Parameters	V	I	Used for
Sphere	36 sectors × 18 stacks, radius 1	$(S{+}1)(K{+}1) = 703$	$6S(K{-}1) = 3672$	nucleus, electrons, atoms
Cylinder	32 segments, radius $\frac{1}{2}$, height 1	$4N{+}6 = 134$	$12N = 384$	bond strands
Torus ×7	$R = r_k$, $\rho = 0.03$, 128×8	$(M{+}1)(m{+}1) = 1161$	$6Mm = 6144$	orbital ring of shell k

The sphere counts are those of the UV sphere of Part 1 (Table 1): the S triangles touching each pole are degenerate and skipped.

Shaders. The vertex shader computes `FragPos = vec3(model * vec4(aPos, 1))`, `Normal = normalMatrix * aNormal` and `gl_Position = projection * view * vec4(FragPos, 1)`; the fragment shader evaluates (2) per pixel.

Listing 2: Fragment shader core (phong.frag).

```
vec3 ambient = light.ambient * material.ambient;

vec3 norm   = normalize(Normal);
vec3 lightDir = normalize(light.position - FragPos);
float diff  = max(dot(norm, lightDir), 0.0);
vec3 diffuse = light.diffuse * (diff * material.diffuse);

vec3 viewDir  = normalize(viewPos - FragPos);
vec3 reflectDir = reflect(-lightDir, norm);
float spec    = diff > 0.0 ? pow(max(dot(viewDir, reflectDir), 0.0),
    material.shininess) : 0.0;
vec3 specular = light.specular * (spec * material.specular);

FragColor = vec4(ambient + diffuse + specular, 1.0);
```

Materials. Unlike Part 1, Part 2 uploads a full `Material` per node. The default light is at $(5, 8, 5)$ with ambient 0.15, diffuse 1 and specular 1 (all editable in *Lighting*).

Object	Ambient	Diffuse	Specular	Shininess
Nucleus	0.3× CPK color	CPK color	$(0.8)^3$	128
Electron	$(0.3, 0.3, 0)$	$(1, 1, 0)$ yellow	$(1)^3$	128
Orbital ring	$(0.05)^3$	$(0.3, 0.4, 0.5)$	$(0.1)^3$	16
Atom (molecule)	0.25× CPK color	CPK color	$(0.6)^3$	80
Bond strand	$(0.15)^3$	$(0.55, 0.55, 0.60)$	$(0.4)^3$	32

The nucleus and atom colors are the CPK colors of the Jmol scheme [15] (e.g. H white, C dark gray, N blue, O red, Cl green); the high shininess of the electrons gives them a small, sharp highlight, the low one of the rings a matte look.

OpenGL state and wireframe. Depth testing, back-face culling (`GL_BACK`, which requires the counter-clockwise winding of Section 4.4.3), 4× MSAA and VSync are enabled; the background is $(0.05, 0.07, 0.12)$. In wireframe mode (`W`) every node's flag is set, the CPU switches `glPolygonMode` to `GL_LINE` for its draw call and the shader outputs the unlit diffuse color, showing the triangulation of the spheres, cylinders and tori.

4.7 User Interface and User Instructions

```
cd part2_molecules
make -j4 && ./build/molecules     # or, from the top folder: make run2
```

Control panel. The panel is fixed in the top-left corner (310 px wide; it scrolls when it is taller than the window) and contains:

- **FPS** and **View Mode**: radio buttons *Bohr Atom Model* / *Ball-and-Stick*; switching re-frames the camera.
- **Bohr Atom Model**: element symbol field (type e.g. `Na` + Enter), list of the 23 elements sorted by Z, name and Z, and for every shell its occupation, its capacity $2n^2$ and its radius; *Animate Electrons* and *Speed* (0.1–5×).
- **Ball-and-Stick Molecules**: list of the 8 molecules; formula, number of atoms and bonds, bond types; *Geometry* (bond lengths and angles); CPK color legend (one line per element); *Edit atoms & bonds* (Section 4.4.5); *Molecular Vibration* with the wavenumber and the displayed frequency, *Amplitude* and *Vib. speed*; *Molecular Rotation*.
- **Rendering**: *Wireframe (W)*. **Lighting**: light position, ambient / diffuse / specular colors. **Camera**: yaw, pitch, field of view, distance, reset.
- **Screenshot (F12)** with the option *with GUI*, and the name of the last saved file.

Input	Action
Left mouse drag	orbit the camera (yaw / pitch)
Right mouse drag	pan (move the target)
Mouse wheel	zoom (change the distance)
`R`	reset and re-frame the camera
`W`	toggle wireframe
`F12` / `Shift+F12`	screenshot of the 3D view / of the whole window
`Esc`	quit

Screenshots are saved as `screenshot_<time>.png` in the current folder. Mouse presses over the panel and keys typed into a text field (e.g. the `R` of "Ar") are not used by the camera.

Screenshots. `F12` reads the 3D viewport from the back buffer *before* the GUI is drawn, `Shift+F12` (or the button with *with GUI*) reads the whole window *after* it; `glReadPixels` returns the rows bottom-up, so they are flipped while the PNG is written.

Command line. All figures of Section 4.9 were produced with commands such as

```
./build/molecules --element Na --screenshot fig.png
./build/molecules --molecule H2O --with-gui --editor --screenshot f.png
```

```
./build/molecules --molecule CO2 --amplitude 0.2 --time 0.1876 \
                  --screenshot fig.png
```

Options: `--element <symbol>`, `--molecule <formula>`, `--wireframe`, `--paused`, `--no-vibrate`, `--rotate`, `--speed <s>`, `--amplitude <A>`, `--time <t>` (advance all animations by t seconds), `--yaw`, `--pitch`, `--distance`, `--light x,y,z`, `--editor`, `--screenshot <file.png>`, `--with-gui`, `--frames N`. With `--screenshot` the animations are frozen at time t, so the images are reproducible.

4.8 Testing and Verification

`make test` (in `part2_molecules`, or at the top level for both parts) builds `tests/test_part2.cpp`, which needs no window: the scenes are built with dummy meshes and only their transforms are inspected. It checks:

- **meshes**: V and I equal the closed forms above, all indices are valid, all normals have unit length, every non-degenerate triangle is counter-clockwise seen from outside ($(\mathbf{b}-\mathbf{a})\times(\mathbf{c}-\mathbf{a})\cdot(\mathbf{n}_a+\mathbf{n}_b+\mathbf{n}_c)>0$), and every torus vertex is at distance ρ from the centre circle;
- **Bohr atoms** (all 23 elements): $\sum_k n_k = Z$ and $n_k \le 2k^2$; at $t=0$, after several time steps, and for an atom built while paused, every electron satisfies $\|\mathbf{c}\| = r_k$ and $\mathbf{c}\cdot\mathbf{n}_k = 0$ (Proposition 3), every ring is rigid and centred on its orbit, the distances of Proposition 4 hold, and one update turns each orbit by exactly $s\,\omega_k\,\Delta t$;
- **molecules** (all 8): bond lengths and angles equal the reference values of Table 5 (± 0.002 Å, $\pm 0.1°$); bond detection finds exactly the preset bonds; positions are the similarity image (5) and the fit radius is reached (Proposition 6); every strand joins its two atoms, with the right radius, a perpendicular offset and both ends inside the balls (Proposition 7); parallel strands do not touch; the normal matrix keeps normals perpendicular to the surface (Proposition 1); during the vibration the centre of mass stays at the origin and all positions are $\lambda(t)\mathbf{x}_i^0$ (Proposition 8); with the rotation everything stays attached;
- **bond placement** for 300 random segments and the degenerate directions $\pm\hat{\mathbf{y}}$.

All 13 607 checks pass. As a control, re-introducing either defect of the previous version (clockwise cylinder triangles, or the transposed axis of the bond rotation) makes the corresponding checks fail.

4.9 Results

All figures are screenshots of the application (default Phong lighting, default camera unless stated), produced with the commands of Section 4.7.

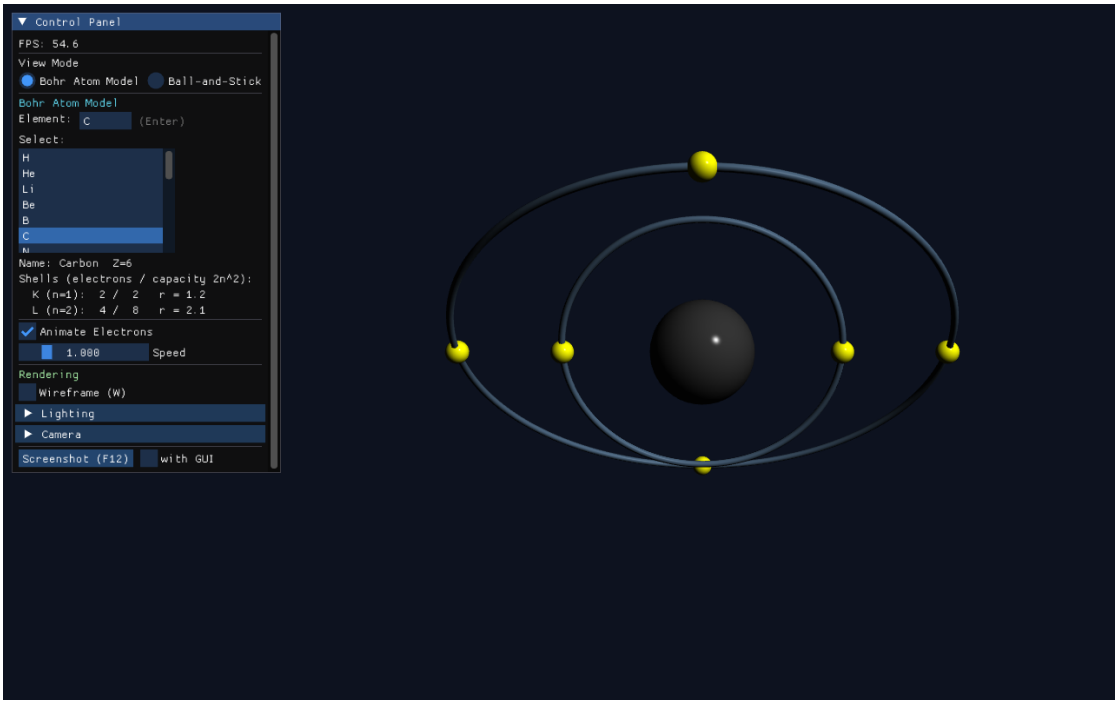
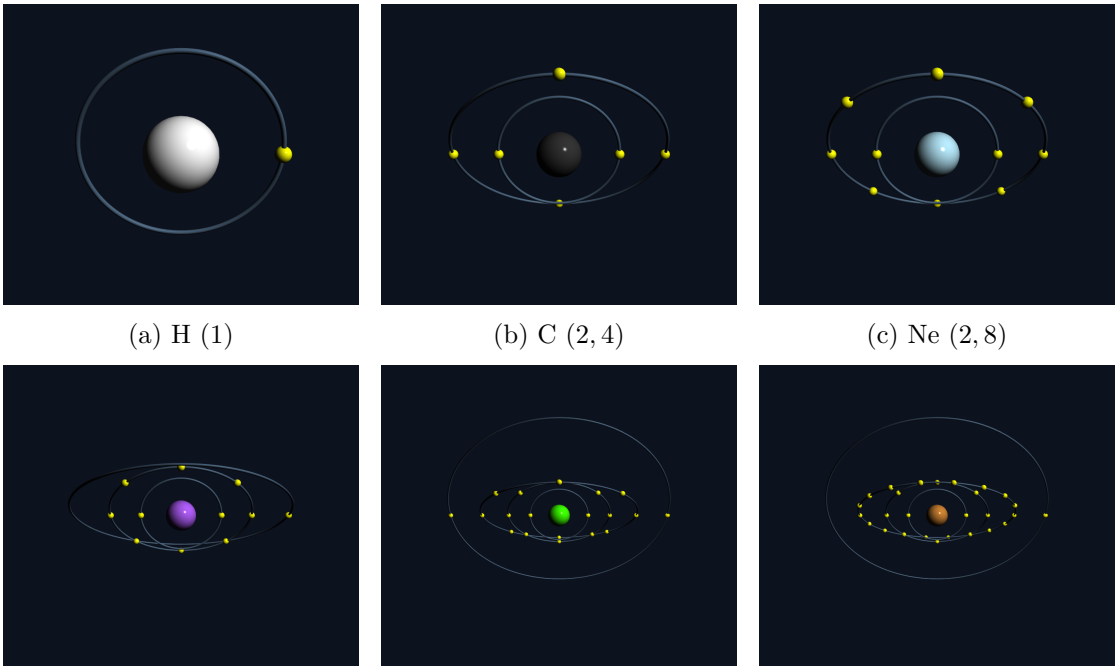


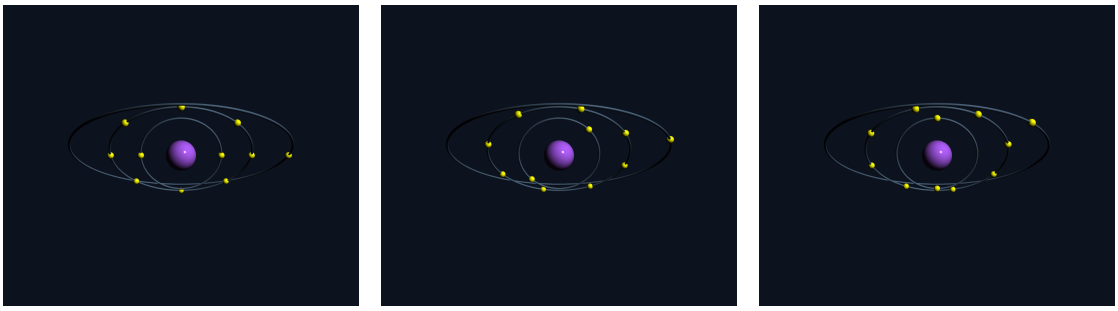
Figure 11: The interface in Bohr mode (carbon): control panel with the element list and the shell occupation (left), 3D view (right).



(a) H (1) (b) C $(2, 4)$ (c) Ne $(2, 8)$

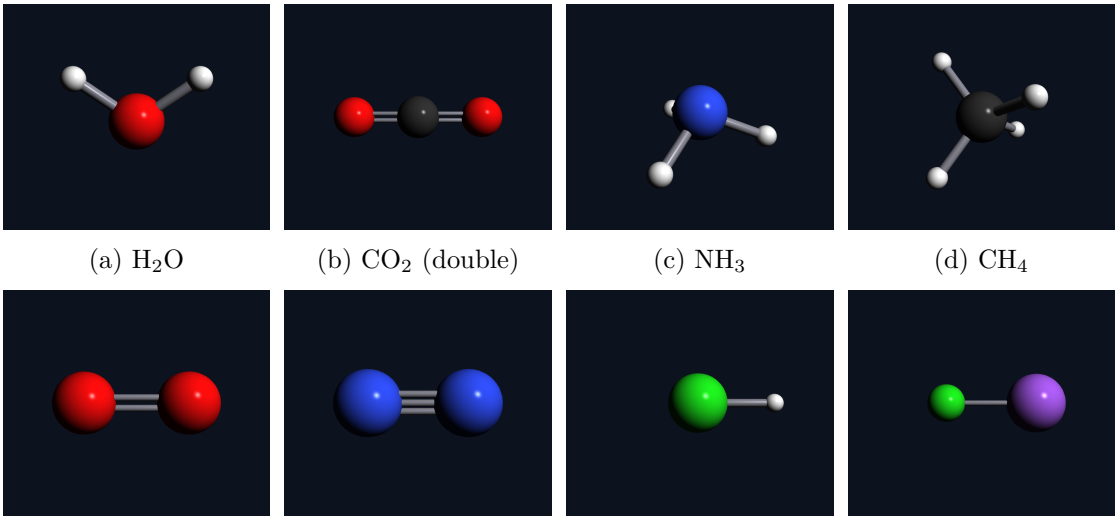
(d) Na $(2, 8, 1)$ (e) Ca $(2, 8, 8, 2)$ (f) Cu $(2, 8, 18, 1)$

Figure 12: Bohr models with one to four shells. The nucleus has the CPK color of the element; each shell is a tilted ring with its electrons evenly spaced; the rings appear as ellipses of different axis ratios (Section 4.3.4).



(a) $t = 0$ (b) $t = 0.5$ s (c) $t = 1$ s

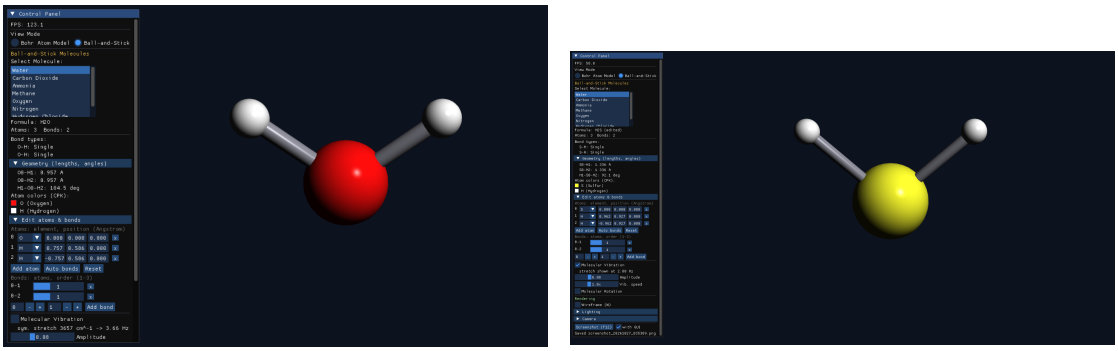
Figure 13: Electron motion (sodium): in one second the K, L and M electrons turn by 90°, 60° and 45° along their rings.



(a) H_2O (b) CO_2 (double) (c) NH_3 (d) CH_4

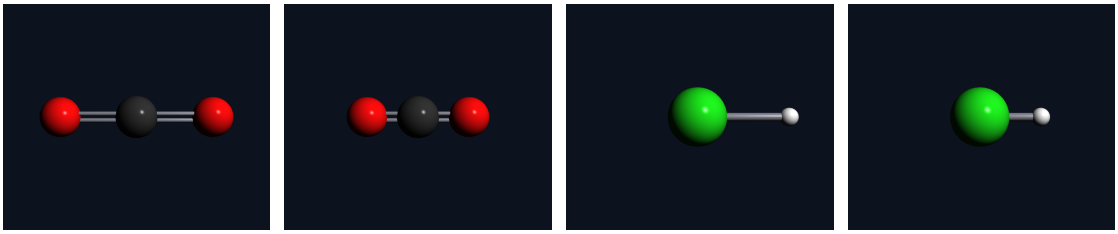
(e) O_2 (double) (f) N_2 (triple) (g) HCl (h) NaCl

Figure 14: The eight ball-and-stick molecules: CPK-colored balls proportional to the covalent radii, bonds of one, two or three strands.



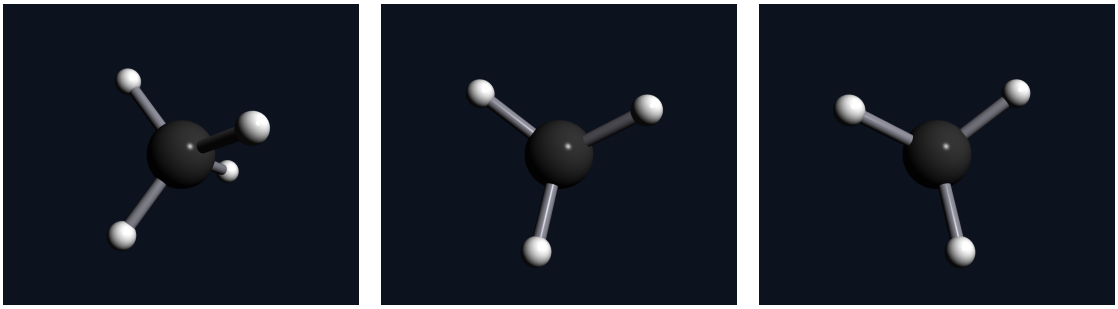
(a) Water: geometry read-out and editor (b) Edited into H_2S: S–H 1.336 Å, 92.1°

Figure 15: Configuration of atom types and positions (Section 4.4.5).



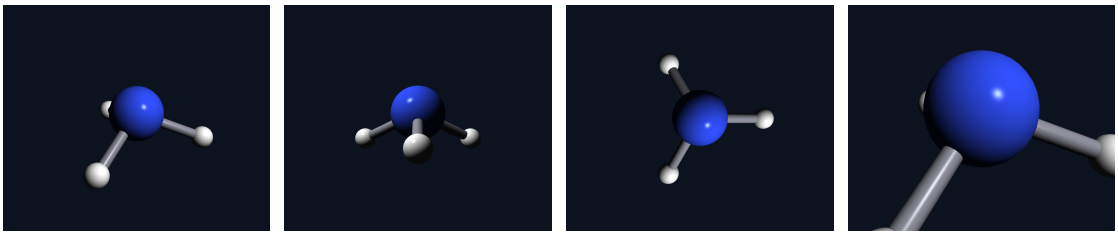
(a) CO_2, $\lambda = 1.2$ (b) CO_2, $\lambda = 0.8$ (c) HCl, $\lambda = 1.2$ (d) HCl, $\lambda = 0.8$

Figure 16: Symmetric stretching with $A = 0.2$ at a quarter and three quarters of a period: the carbon of CO_2 and, almost, the chlorine of HCl stay at rest (centre of mass), the bonds stretch with the atoms (Proposition 8).



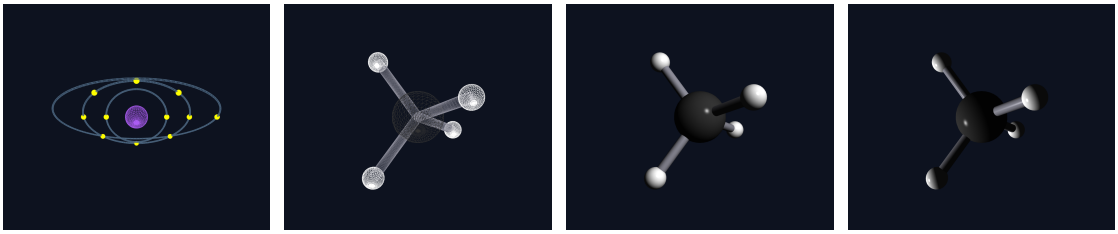
(a) $t = 0$ (b) $t = 1$ s (30°) (c) $t = 2$ s (60°)

Figure 17: Rotation of methane about the vertical axis through its centre of mass.



(a) default view (b) side ($\psi = 0$, $\varphi = 0$) (c) top ($\varphi = 89°$) (d) zoom ($d = 3.5$)

Figure 18: Camera control on ammonia: the side view shows the pyramid, the top view (along the threefold axis) the 120° symmetry of the H atoms.



(a) Na, wireframe (b) CH_4, wireframe (c) light $(5, 8, 5)$ (d) light $(-10, -2, 4)$

Figure 19: Wireframe mode (triangulation of the shared meshes) and the effect of the light position on the Phong shading.

4.10 Limitations

- Bohr's model is semi-classical: electrons do not move on circles but occupy orbitals (probability clouds), and the model is quantitatively valid only for one-electron atoms. The rings, their tilts and the compressed radii and speeds are a visual convention (Section 4.3.2), not to scale. Only 23 elements are in the table (up to four shells).
- Molecules come from 8 presets that the editor can modify; there is no import of molecular files (`.xyz`, `.mol`) and no saving of edited molecules. Bond orders are not inferred. The molecule is re-centred and re-scaled after every edit, so it may appear to change size while an atom is dragged.
- Geometries are fixed experimental values; there is no force field or geometry optimisation. The vibration is the symmetric stretch only (exact for diatomics, CO_2 and CH_4, approximate for H_2O and NH_3); bending and asymmetric modes would need the normal-mode analysis of Section 4.5.
- Atom labels are not drawn in the 3D view; the CPK legend and the geometry read-out of the panel identify the atoms.

4.11 Limitations

- Only 23 elements are included in the look-up table; elements beyond Zn are not available. The molecule list is fixed at compile time (8 molecules); the user cannot define new molecules at runtime.
- Bohr's model is a simplification: in reality electrons do not orbit in fixed circular paths but occupy probability clouds (orbitals). The tilted-ring visualization is a pedagogical convention, not a physical representation.
- Bond lengths and angles are hard-coded from approximate experimental data; no force-field relaxation or geometry optimization is performed.

- The ball-and-stick model uses the molecule mode that is currently commented out in the GUI radio buttons (the code is complete but the UI toggle to switch to it is disabled in the delivered version). Users would need to uncomment the radio button in `main.cpp` (lines 187–191) to enable it.
- There is no screenshot function in Part 2 (unlike Part 1 which uses `stb_image_write`). Screenshots must be taken with an external tool.
- The vibration animation is a simplified sinusoidal oscillation along a single axis per atom, not a physically accurate normal-mode analysis.
- Atom labels (element symbols displayed next to each sphere) are not rendered; the CPK color legend in the GUI is the only way to identify atoms.

5 Conclusion

Both applications are built on the same OpenGL 3.3 core-profile foundation and on the shared code in `common/` (shader loading, orbit camera, ImGui, GLM), so the two parts are independent programs that reuse the same graphics infrastructure.

5.1 Part 1

Part 1 draws all required 2D and 3D shapes, a user-defined surface and imported `.obj` / `.ply` models; supports the five render modes (flat, Gouraud, Phong, texture, wireframe); lets the user translate, rotate and scale every object and navigate with mouse and keyboard; and is organised in small modules of which the geometry, parsing and loading parts are independent from OpenGL.

5.2 Part 2

Part 2 visualizes atoms with Bohr's model (23 elements, one tilted ring per shell, electrons that provably stay on their rings and never overlap) and molecules with the ball-and-stick model (8 molecules with experimental geometry, CPK colors, balls proportional to the covalent radii, bonds of one to three strands placed by an exact axis–angle construction). It lets the user configure the element of the atom and the type and position of every atom of a molecule, with automatic bond detection and a geometry read-out. It animates the electron motion, a symmetric stretching vibration that conserves the centre of mass and the momenta, and a rotation about the centre of mass, and it frames the camera automatically and saves screenshots. The scene graph composes all transformations, the meshes are shared, every node has its own material under per-fragment Phong lighting.

6 Source Code

https://github.com/myphuong0309/261-computer-graphics-assignment1 (folders `part1_shapes`, `part2_molecules` and `common`).

References

[1] M. Segal and K. Akeley, *The OpenGL Graphics System: A Specification, Version 3.3 (Core Profile)*, Khronos Group.

[2] B. T. Phong, "Illumination for Computer Generated Pictures," *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.

[3] H. Gouraud, "Continuous Shading of Curved Surfaces," *IEEE Transactions on Computers*, vol. C-20, no. 6, pp. 623–629, 1971.

[4] GLFW – an OpenGL library for windows, input and contexts. https://www.glfw.org

[5] GLEW – the OpenGL Extension Wrangler Library. https://glew.sourceforge.net

[6] G-Truc Creation, OpenGL Mathematics (GLM). https://github.com/g-truc/glm

[7] O. Cornut, Dear ImGui – bloat-free graphical user interface for C++. https://github.com/ocornut/imgui

[8] S. Barrett, stb single-file public domain libraries (`stb_image`, `stb_image_write`). https://github.com/nothings/stb

[9] N. Bohr, "On the Constitution of Atoms and Molecules," *Philosophical Magazine*, Series 6, vol. 26, no. 151, pp. 1–25, 1913.

[10] Wikipedia, *Bohr model*. https://en.wikipedia.org/wiki/Bohr_model

[11] Wikipedia, *Ball-and-stick model*. https://en.wikipedia.org/wiki/Ball-and-stick_model

[12] E. B. Wilson, J. C. Decius and P. C. Cross, *Molecular Vibrations: The Theory of Infrared and Raman Vibrational Spectra*, McGraw-Hill, 1955 (Dover reprint, 1980).

[13] Wikipedia, *Molecular vibration*. https://en.wikipedia.org/wiki/Molecular_vibration

[14] Wikipedia, *Infrared spectroscopy*. https://en.wikipedia.org/wiki/Infrared_spectroscopy

[15] Jmol, *Jmol Colors* (CPK color scheme). https://jmol.sourceforge.net/jscolors/